



*Algerian Democratic and Popular Republic Ministry of
Higher Education and Scientific Research
University of Hassiba Benbouali Chlef
UHBC*



*Faculty of Sciences
Department of Computer Science
Master of Science Thesis*

Option

Software Engineering

**Design and modeling multi-core NUMA simulator and
protocols**

Supervisor :

Mr. M. SLIMANE

BY:

Abed MAATALLA

Jury members :

- Mr. M. ARIDJ
- Mr. A. LOUAZANI
- Mr. M. SLIMANI

Year 2014/2015

Acknowledgments.

I would like to express my gratitude to my supervisor Mr.: Slimane: Mohamed for introducing me to this area of research and for providing me with guidance and support. I would also like to thank the scientific jury chief Mr: A. SAIH and jury members Mr: A. LOUAZANI and Mr: R. GHERBAOUI.

Also to every professor at Hassiba: Benbouali: University in Chlef who have been the reason for my success and a bridge to achieve this point that I'm proud of. I'm grateful of being a part of GDG:Chlef:Community (Google Developer Groups) that has endorsed my passion about technology and mostly Google technologies. So thank to everyone within this community, I will mention Ilyes:Boudjelthia for being just awesome, Dellal:Mohamed for being a hard worker and Yasser Cherifi for its motivations and last but not least Bilal:Sefsaf for being a good friend, knowing you all is a blessing.

One other special thank I want to give to Algeria's number one geek Mohamed:Amine Aboura who had always answers to my questions, I'm glad to have you as a friend.

Saving the best for the last, I want to thank my mom for her prayers and for preparing coffee on the go for me, and to my dad for the support and wise words

Figure 1.1: Moore's law showing increasing number of transistors per year-----	4
Figure 2.1: Design choices for multi-core processors -----	6
Figure 3.1: Quad-Core AMD Opteron VS Quad-Core Intel Xeon -----	7
Figure 4.1: NUMA node -----	9
Figure 5.1: A Simple NUMA Architecture-----	10
Figure 6.1: Eight core Multi-socket Nehalem Processor-----	11
Figure 7.1: Remote access memory-----	12
Figure 8.1: Memory Hierarchy -----	13
Figure 9.1: Data transport between cache and main memory -----	14
Figure 10.1: A Simple Symmetric Multiprocessing System-----	17
Figure 11.1: Directory base Protocol Schema -----	18
Figure 12.2: a simple analytic module-----	21
Figure 13.2: Model Taxonomy-----	23
Figure 14.2: GRAPHIT Simulator Architecture -----	25
Figure 15.3: A simple NUMA architecture topology -----	28
Figure 16.3: NUMA topology generated by HwLoc-----	29
Figure 17.3: Illustration of an application which uses libvorbisfile-----	32
Figure 18.3: Homogeny of sockets-----	35
Figure 19.3: Package Diagram modeling -----	36
Figure 20.3: Interface Structure Diagram-----	36
Figure 21.3: NUMA Simulator Class Diagram -----	37
Figure 22.3: An overview of NUMA class diagram -----	38
Figure 23.3: sequence diagram of local memory access to cache. -----	39
Figure 24.3: sequence diagram of local memory access to RAM. -----	39
Figure 25.3: sequence diagram of remote memory access. -----	40
Figure 26.3: simulation process of executing a task on a single core-----	40
Figure 27.3: Class diagram of basic simulator library-----	42
Figure 28.3: Scheduler process interaction -----	44
Figure 29.4: example des Implements des interfaces. -----	48
Figure 30.4: First interconnection class -----	48
Figure 31.4: Second interconnection class-----	49
Figure 32.4: IHierarchical class interface-----	49
Figure 33.4: addNumaEventListener to fire events from component. -----	50
Figure 34.4: INumaEventListener to handle events and pass it to listener. -----	50
Figure 35.4: events fire in each resources-----	51
Figure 36.4: JAVASim listening to NUMA machine event fire-----	52
Figure 37.4: example of Class is listening to events -----	52
Figure 38.4: Machine loader -----	53
Figure 39.4: data mapping & tasks scheduling -----	54
Figure 40.4: NUMA simulator workflow -----	55
Figure 41.4: topology of two nodes and two cores-----	55
Figure 42.4: mapping data -----	56
Figure 43.4: scheduling tasks -----	56
Figure 44.4: Event fire record in time as plot chart -----	57
Figure 45.4: Gantt diagram[17] showing the schedule of events -----	57
Figure 46.4: topology of four nodes and two cores -----	58
Figure 47.4: Mapping data-----	58
Figure 48.4: scheduling tasks -----	59
Figure 49.4: Event fire record in time as plot chart -----	60
Figure 50.4: Gantt diagram[17] showing the schedule of events -----	60

INDEX

Abstract.....	01
Introduction.....	02

Chapter One: Multi-core and NUMA Systems

1) multi-core Processors.....	05
1. 1) Overview.....	05
1. 2) multi-core Architecture.....	05
1. 2. 1) Hierarchical Design.....	06
1. 2. 2) Pipelined Design.....	07
1. 2. 3) Network Based Design.....	07
2) NUMA Systems.....	08
2. 1) Basic Issue.....	08
2. 2) ccNUMA.....	09
2. 3) NUMA Architecture.....	09
2. 3. 1) UMA VS NUMA.....	09
2. 3. 2) Hardware Implementation.....	10
2. 4) Interconnections.....	12
2. 5) Remote memory access.....	12
3) Memory Hierarchy.....	13
3. 1) Characteristics of Caches.....	13
3. 1. 1) CacheSize.....	14
3. 1. 2) Mapping of Memory Blocks.....	14
3. 2) Locality.....	15
3. 2. 1) Spatial Locality.....	15
3. 2. 2) Temporal Locality.....	15
3. 3) Shared Memory.....	16
3. 3. 1) Symmetric Multiprocessing.....	16
3. 3. 2) Cache Coherence.....	17
3. 3. 2. 1) Snooping protocol	18
3. 3. 2. 2) Directory-based Protocols	18
4) Summary.....	19

Chapter Two: Multi-core & NUMA simulators

1) Simulator.....	20
1. 1) Analytic simulation	20
1. 2) Discrete event simulator	20
2) Multi-core & NUMA simulator benefits	21
3) Simulation.....	22
3. 1) System.....	22
3. 2) Model.....	22
3. 3) Why model?.....	22
3. 4) Characterizing a model.....	22
3. 4. 1) Static & dynamic	22
3. 4. 2) Deterministic & stochastic	22
3. 4. 3) Continuous & discrete	22
3. 5) DES discrete event simulation	22

3. 6) Why simulator for NUMA?	24
4) Current simulators	24
4. 1) Graphite	24
4. 2) Mcocresin	25
4. 3) Gen5 ruby	25
4. 4) simNUMA	25
5) Summary	26

Chapter Three: Modeling and Design

1) Introduction.....	27
2) NUMA machine topology.....	27
2. 1) Hardware topologies.....	27
2. 1. 1) Definition.....	27
2. 1. 2) NUMA topology.....	28
2. 2) HwLoc.....	28
2. 2. 1) DTD of xml file.....	29
3) High level NUMA simulator.....	31
3. 1) Why high level NUMA simulator?.....	31
3. 2) What are the need of high level NUMA simulator.....	31
3. 3) Structure of our simulator.....	31
3. 4) Simulator design	31
3. 4. 1) Separation between model and simulator.....	31
3. 4. 2) Static libraries.....	32
3. 4. 3) Shared libraries.....	32
3. 5) Design approach.....	32
3. 6) Design and modeling process.....	33
3. 7) How we design the NUMA model.....	33
4) Model hypotheses	34
5) Class diagram.....	35
6) Sequence diagram	39
7) Simulation approach	41
7. 1) Introduction.....	41
7. 2) Existing simulator.....	41
7. 3) Basic simulation object.....	41
7. 3. 1) Simulation object.....	41
7. 3. 2) Event manager object.....	41
7. 3. 3) Entity object.....	41
7. 3. 4) Process.....	41
7. 4) JAVASim.....	42
7. 4. 1) Why JAVASim simulator.....	42
7. 4. 2) How JAVASim works.....	42
7. 4. 3) Entities basic methods	43
8) Simulation and model relationship.....	44
8. 1) Processors placement on scheduler queue.....	44
8. 2) Scheduler class interface.....	44
8. 3) Simulation processes	45
8. 4) Illustration	45
9) Summary.....	46

Chapter Four: Implementation and test

1) Introduction.....	47
2) NUMA machine implementation.....	47
2. 1) Interfaces	47
2. 2) Extensibility on model using interfaces.....	48
2. 3) Hierarchical.....	49
3) Events based model.....	50
4) Simulator integration.....	51
5) Topology loader.....	52
6) Task & Data.....	53
7) Simulation scenario.....	54
8) Experiments.....	55
8. 1) Scenario01.....	55
8. 2) Scenario02.....	58
9) Summary.....	61
Conclusion	62

Bibliography

[1] Blagodurov, S., Dashti, M., & Fedorova, A. (2012). A Case for NUMA aware Contention Management on Multicore Systems. Burnaby: Simon Fraser University.

[2] Georg, H., & Gerhard, W. (2011). Introduction to High Performance Computing for Scientists and Engineers. Berkeley: CRC Press.

[3] Goglin, B., & Furmento, N. (2009). Memory Migration on Next Touch. Bordeaux: University of Bordeaux.

[4] Hesham, E. R., & Mostafa, A. (2005). Advanced Computer Architecture And Parallel Processing. Kuwait: Wiley Interscience.

[5] Jean Loup, B. (2010). Microprocessor Architecture. Seattle: Cambridge.

[6] Joakim, N. (2011). Simulating NonCUniform Memory Access Architecture for CloudServer. Abo Akademi: Abo Akademi University.

[7] Manchanda, N., & Anand, K. (2001). Non Uniform Memory Access. New York: New York University.

[8] Martin, B., & Matt, D. (2004). Linux on NUMA Systems. Ontario: Linux Symposium.

[9] Ros, A., Acacio, M., & MarceloCintra. (2011). A Novel Mapping Policy for Distributed Shared Caches. Murcia: Universidad de Murcia.

[10] Nathan Binkert¹, Bradford Beckmann², & all authors, The gem5 Simulator

[11] Jason E. Miller & all authors Graphite: A Distributed Parallel Simulator for Multicores, Massachusetts Institute of Technology, Cambridge, MA

[12]* Sunil Kumar, Tommaso Cucinotta and Giuseppe Lipari, A Latency Simulator for Many-core Systems (McoreSim)

[13]* <http://www.open-mpi.org/projects/hwloc/>

[14]* Yi Liu, Yanchao Zhu, Xiang Li, Zehui Ni, Tao Liu, Yali Chen, Jin Wu, (2013) SimNUMA: Simulating NUMA-Architecture Multiprocessor Systems Efficiently

[15]* <https://github.com/open-mpi/hwloc/blob/master/hwloc/hwloc.dtd> 2015

[16] <http://www.jfree.org/>

[17] <http://swiftgantt.sourceforge.net/>

Abstract

Over the last years, multi-core processors have proven to be a trend even for general purpose computing systems, by achieving a significant performance increase compared to single core processors. Therefore, the problem of scheduling in such systems is one of the hottest and the most challenging topics in computers research. The multi-core architecture usually includes NUMA (Non-Uniform Memory Access) which further complicates this problem, as on NUMA the scheduler needs not only to consider the placement of threads on cores, but also the placement of the data on memory across NUMA nodes.

The research in our master thesis focuses on designing and modeling multi-core NUMA simulator to create different topologies of NUMA architectures. In addition to see how NUMA system behavior on different scenario. **Our main goal is to create a framework that will simulate a real multi-core NUMA machine and its protocols.** Eventually, respecting software engineering roles in process of development will result readable, an easy to use and maintainable code.

Keywords: Multicore Processor, Design pattern, NUMA, HwLoc, Hierarchical technique, Observer / Observable.

Introduction

Computers with multi-core processors have recently become a trend, and the area of use of these CPUs has exceeded servers and workstations to general purpose computers. This growing technology has improved the user experience not only with PCs but also other devices like smart phones, tablets and game consoles. This growth of technology have been already observed in 1965 by Gordon Moore[1] when he deduced that the number of transistors that can be cost effectively built on a chip were going to double every year, but around 1970 this assumption was originated to every two years and the term Moore's Law was coined.

This increasing complexity of microprocessors has eventually led to computers with higher performance and processing capacity. While hardware technology is moving at a rapid pace, software and programming models have failed to keep up with it. For example in 2007, Intel has announced in the International Solid-State Circuits Conference that they have reached 80 cores in TeraFlops Research as known as Polaris, but it has set time of 5 years to make this processor generally available due to the inability of current operating systems and software to exploit the benefits of such multi-core architecture for researching, data-mining and artificial intelligent. Therefore, any hardware evolution must be followed by software updates to achieve a fully optimized system.

Background

Simulators can be used for 'Human in the Loop' (1) or for 'Hardware in the Loop' (2) operation. In both cases the simulator tries to recreate the environment which is relevant to the subject and the purpose of the simulator. This means that the requirements of a simulator differ from application to application and require careful selection of the components.

Simulation software is based on the process of modeling a real phenomenon. It is, essentially, a program that allows the user to observe an operation through simulation without actually performing that operation. Simulation software is used widely to design equipment so that the final product will be as close to design specs as possible without expensive in process modification. When the penalty for improper operation is costly, such as processors, airplane pilots, nuclear power plant operators, or chemical plant operators, giving valuable training experience without fear of a disastrous outcome.

Advanced computer programs can simulate power system behavior such as multi-core NUMA architectures and its topologies. Simulation can be difficult because most natural phenomena are subject to an almost infinite number of influences. One of the tricks to developing useful simulations is to determine which the most important factors that affect the goals of the simulation.

(1) Human-in-the-loop or HITL is defined as a model that requires human interaction.

(2) Hardware-in-the-loop (HWIL) is a technique that is used in the development and test of complex real-time embedded systems.

In addition to imitating processes to see how they behave under different conditions, simulations are also used to test new theories. After creating a theory of causal relationships, the theorist can codify the relationships in the form of a computer program. If the program then behaves in the same way as the real process, there is a good chance that the proposed relationships are correct.

Problem Statement

The primary problem is to design a simulator that can respect as many as possible the real NUMA architectures by modeling simulator that could create different topologies, easy to maintain, flexible and extensible.

Designing NUMA simulator is one of the big challenging problem because hardware is very wide differ from manufacture to another in multi-level cache, multi-level channels, interconnection specifications such as bandwidth, routing algorithms and buffering and many other topologies.

Trying to modeling such kind of architecture by respect software engineering roles along with different other problems that is facing NUMA today and increasing of cores in NUMA systems turns to be very challenging.

The research in our thesis focuses on finding effective ways to simulate different NUMA multi-core architecture topologies and focuses on modeling a complete NUMA machine that implements multiprocessing hierarchical platforms, and redesign it to be aware of NUMA multi-core architectures topology. In other word, more specifically the aim of this research is to create fully easy improvable, reintegrate **library dedicate for NUMA system**.

Thesis Structure

This thesis report is structured as follows:

Chapter one presents the common concepts of multi-core platforms and NUMA architectures and we focus on its hierarchical natures, performance characteristics and its challenges.

In Chapter Two, we describe the structure of simulator and the most important benefits of simulators. Also as while as the most recent simulators that have been implemented.

In Chapter Three, we will see the modeling and design process throw different diagrams, hypostasis and challenges we are facing.

Finally, the fourth chapter, we will view together the results and chart that our model is providing.

For the last few decades Moore's Law has continually provided exponential growth in the number of transistors on a single chip, this enormous increase has enabled hardware manufacturers to provide a significant performance increase. For many years, the most important factors for the performance increase have been an increase in clock speed (1) of the processor, but this traditional technique has mainly reached its limits. There are two main reasons why the speed of processor clocks cannot be increased significantly. First, the increase in the number of transistors on a chip increases the power density and heat production, and the second reason is that memory access time could not be reduced at the same rate as the processor clock period. (Thomas & Gudula, 2010)

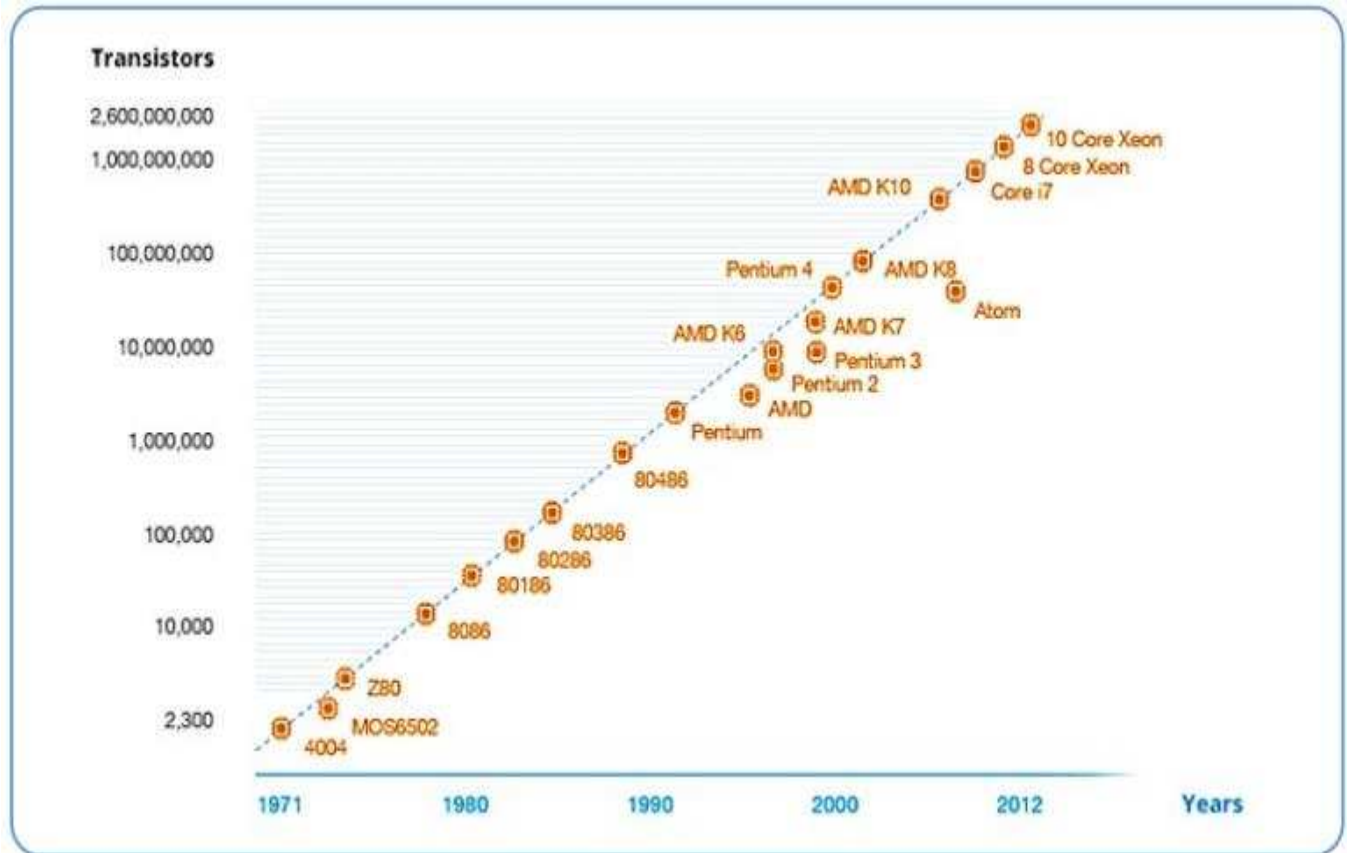


Figure 1.1: Moore's law showing increasing number of transistors per year

In fact, processor designers had to deal with even more problems to keep the rate of the previous performance increase, prompting them to consider new processor architecture, and the use of multiple cores on a single processor seemed to be the most promising approach. This chapter is a quick overview of the multicore architecture used in recent computers. [2]

(1) CLOCK SPEED: is operating speed of microprocessors expressing cycles per second measured by Megahertz.

1. Multi-core Processors

1. 1. Overview

Multi-core processors integrate multiple execution cores on a single CPU chip. As for the OS, each execution core represents an independent logical processor with separate execution resources. Each core has to be controlled separately, and the OS can assign different application programs to the different cores to obtain a parallel execution. Background applications like virus checking or image compression can run in parallel to application programs of the user.

By using techniques of parallel programming such as Multi-threading (1) and Symmetric Multiprocessing, it is also possible to execute a computation intensive application programs like scientific calculations in parallel on a set of cores to reduce the execution time and it may also lead to more accurate results by performing more computations as in the sequential case. In the future, users will likely expect an efficient use of the execution cores of a processor. To achieve this, programmers in turn must consider the new computing paradigm, and the use of parallel programming techniques is highly required. These techniques are to be discussed later, but before that it is recommended to know more about multi-core processors.

1. 2. multi-core Architecture

There are many different design variants for multi-core processors, differing in the number of cores, the structure and size of the caches, the way core access to caches, and the use of heterogeneous components. From a high level view, three different types of architectures can be distinguished, and there are also hybrid organizations. (Kogge, 2005) In the section that follows we describe these different types of multi-core architectures. In Figure 1.2 we see three illustrations of these types, from left right we distinguish: Hierarchical Design, Pipelined Design and Network Based Design.

- (1) Multi-core: used to reduce access time by simulating a number of virtual processors for each physical processor

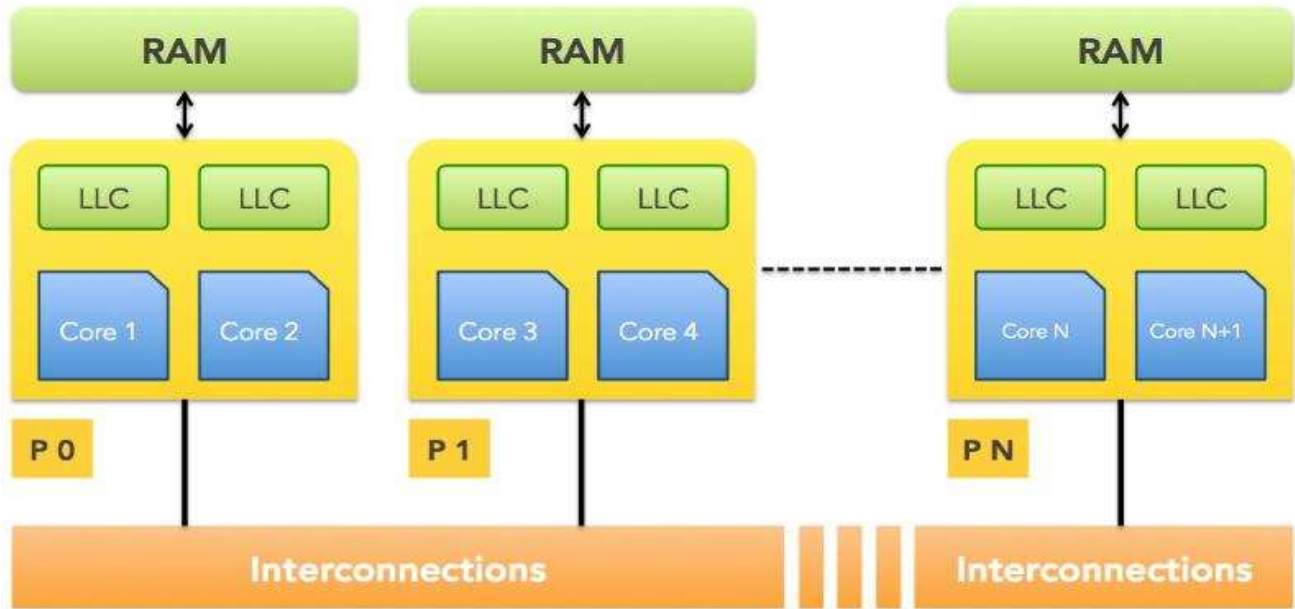


Figure 2.1: Design choices for multi-core processors

1. 2. 1. Hierarchical Design

For a hierarchical design, multiple cores share multiple caches. The caches are organized in a configuration like a tree, and the size of the caches increases from the leaves to the root. The root represents the connection to external memory. Thus, each core can have a separate L1 cache and shares the L2 cache with other cores. All cores share the common external memory resulting in a Three-Level hierarchy as illustrated in Figure 1.2 on the left. A typical usage area for a hierarchical design is the SMP configuration.

This hierarchy is often used for standard desktop or server processors. We take an example from the two most known in microprocessor industry, the processors of Intel Xeon and AMD Opteron. The architecture of these two processors is illustrated in Figure 1.3 as a typical example for desktop CPU with a hierarchical design.

Also many Graphics Processing Units (GPUs) exhibit a hierarchical design, for example the Nvidia GeForce 9 Series, which has around 128 stream processors, these stream processors can be considered as simplified cores that perform a single thread at a very slow rate. Such architecture can be scalable to smaller and larger configurations by scaling the number of SPs and memory partitions. Recently in 2014 during GPU Technology Conference, Nvidia has announced the new GeForce GTX Titan Z featuring dual GPUs, each with 2880 CUDA [3] cores giving the power of 5760 cores. Unlike traditional dual GPU cards, Titan Z GPU twins are tuned to run at the same clock speed so neither GPU creates a performance bottleneck.

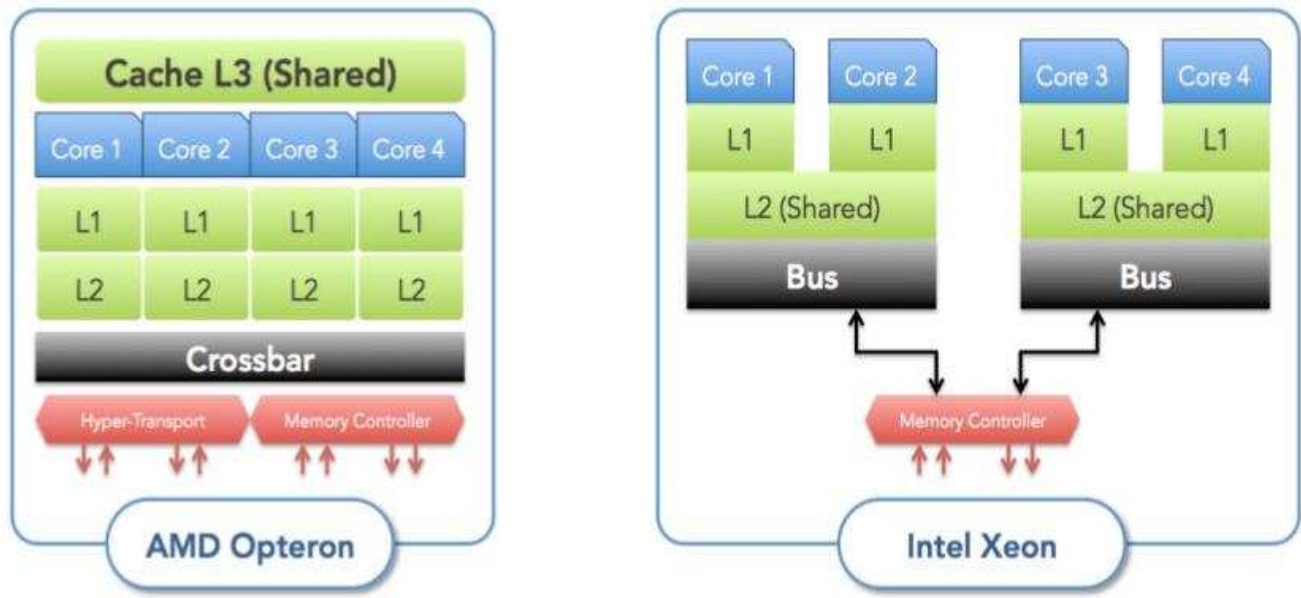


Figure 3.1: Quad-Core AMD Opteron VS Quad-Core Intel Xeon

1. 2. 2. Pipelined Designs

For a pipelined design, data elements are processed by multiple execution cores in a pipelined way. Data elements enter the processor chip via an input port and are passed successively through different cores until the processed data elements leave the last core and the entire processor chip via an output port. Each core performs specific processing steps on each data element, see Figure 1.2 in the middle.

1. 2. 3. Network base design

For a network-based design, the cores of a processor chip and their local caches and memories are connected via an interconnection network with other cores of the chip, as shown in Figure 1.2 on the right. Data transfer between the cores is performed via the interconnection network. This network may also provide support for the synchronization of the cores. Off-chip interfaces may be provided via specialized cores or DMA [4] ports.

A familiar example for a Network-Based design is the Intel Teraflop processor, which has been designed by the Intel Tera-Scale Computing Research Program. This program addresses the challenges of building processor chips with tens to hundreds of execution cores, when the actual developed prototype contains 80 cores. These challenges include the design of processor cores, energy management, cache and memory hierarchy. This last will be discussed in the next titles but before that, here's some examples of multi-core processors in 2009.

Processor	Cores #	Clock	L1 Cache	L2 Cache	L3 Cache
Intel Xeon E5450 (Harpertown)	4	3.0 GHz	4x32 KB	2x 6.1 MB	/
Intel Xeon E5540 (Gainestown)	4	2.5 GHz	4x64KB	4x256MB	8MB
AMD Opteron (Barcelona)	4	2.0 GHz	4x64KB	4x512MB	2MB
AMD Opteron (Istanbul)	6	2.8GHz	6x128KB	6x512MB	6MB
IBM Power 6	2	4.7 GHz	128KB	2x4MB	32MB
Sun T2 Niagara 2	8	1.2GHz	8x8KB	4MB	/

Table 1: Examples for multi-core processors

2. NUMA Systems

2. 1. Basic Issue

The increasing use of parallelism on almost all kind of computers has unleashed a massive computation power not only by incrementing the number of cores in a chip but also due to clock speed improvement of these cores, but on the other side, memory such SRAM was unable to serve the processors at the same speed as result of memory access constraints in such kind of memory. This problem is referred to as memory wall problem, and because of this problem, microprocessor designers were urged to find a less affordable solution by building more high speed memory on the same chip but with less and different capacities. The integration of a sequence of memory on a single chip along with processor cores generates a case known as Non-Uniform Memory Access (NUMA) where memory access can vary depending on the position of the memory module and the core trying to access it.

Placing memory modules spatially nearby the accessing core reduces memory latencies but further increases the complexity of multi-core architectures. Developing for many core architectures with on-chip networks and memory that exhibit NUMA puts additional requirements on the runtime environment or the application programmer. These new challenges imposed by NUMA architectures need to be handled by either extending the computer system or the programming model. (Lof & Holmgren 2005)

In this work we attempt to come up with strategies to avoid costly penalties of NUMA accesses by considering task and data distribution. We desire minimizing the cost of accessing the memory by achieving affinity between tasks and data, and as result minimizing overall execution time of the application program.

2. 2. ccNUMA

The hardware trend is to use NUMA systems with several NUMA nodes as shown in figure. A NUMA node has a group of processors having shared memory. A NUMA node can use its local bus to interact with local memory. Multiple NUMA nodes can be added to form a SMP. A common SMP bus can interconnect all NUMA nodes.

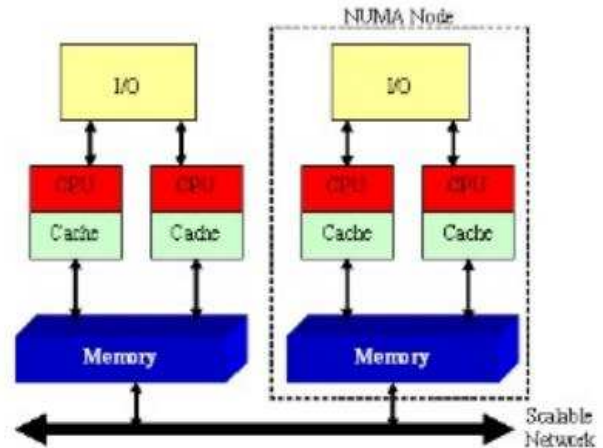


Figure 4.1: NUMA node

2. 3. NUMA Architecture

To fairly understand the architecture of NUMA systems and the benefits it offers we need to compare it to other architectures based on Uniform Memory Access. These architectures were designed to handle a lower number of processors or these processors can handle only certain tasks. As the number of processors increase and the trend of multiprocessors systems these architectures began showing performance and scalability issues.

2. 3. 1. UMA VS NUMA

Parallel computing would not be achieved if not the use of shared memory where parallel processes share a common address space. The distance between the main memory and all processors in the system can be identical, in this case we have Uniform Memory Access, or it can vary as a consequence of distributing of the main memory along with each processor, and in this case we have Non-Uniform Memory Access.

The UMA organization is often referred to Symmetric Multiprocessing (SMP) where the interconnection is a common shared bus as shown in Figure 1.6 above. Although there is no need for the processors to be homogeneous as long as they have the same ISA [5] and although the caches can be of various geometries, this organization has offered performance increases in many situations, even when applications were written for a single processor system. However, the use of a shared bus limits the number of processors in an SMP for at least two reasons, electrical constraints on the load of the bus, and contention for bus access, and so the alternative is to have Non-Uniform Memory Access.

As for NUMA organization, each processor has access to all available memory, same case as SMP. However, each processor is directly connected to a portion of the available physical memory. In order access other parts of the physical memory, it needs to use a fast interconnection link. Accessing the directly connected local memory is faster than accessing the remote memory, as shown in Figure 1.8 bellow. The OS and applications must be written having in mind this specific behavior that can affects the performance. (Lakovos, 2011)

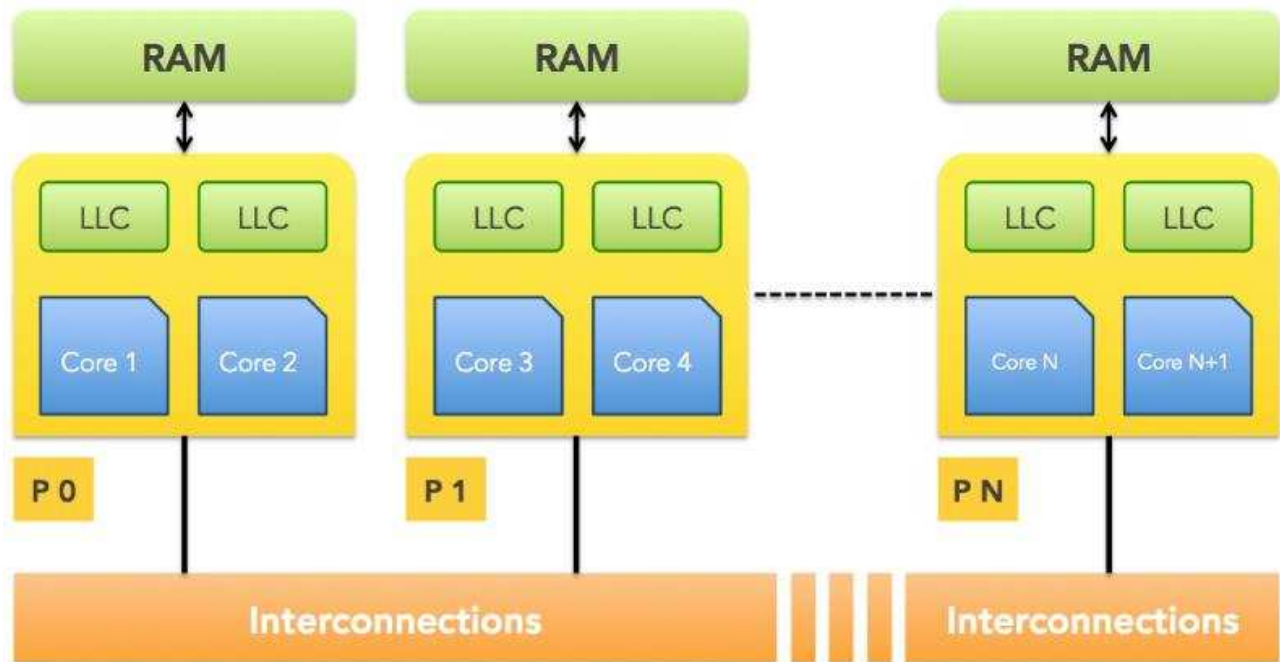


Figure 5.1: A Simple NUMA Architecture

2. 3. 2. Hardware Implementation

NUMA Architecture has unlocked the possibility of building motherboards that support multi-socket hardware platform to attain a developed multiprocessors system, where each processor hierarchy can be multi-core itself as illustrated in Figure 1.8 above. Each processor can access any data elements across the shared global memory, as for the local internal memory that we call cache, it is only accessible by the processor where it is located. In such situations, the problem of cache coherence can extend to NUMA Systems.

However, most NUMA systems are cache coherent and they are formally called ccNUMA platforms. Writing operating systems and applications for ccNUMA is easier and is similar to writing applications for SMP systems. ccNUMA platforms use special hardware that enables inter-processor communication between the cache controllers. For every cache miss (read or write), every processor must communicate with all the others processors in order to make sure that the next fetch from main memory is in a current state and not stored in another cache. This communication has a significant overhead. In order to reduce this overhead we need to avoid access to the same memory area by multiple processors. Furthermore, NUMA platforms use specialized cache coherency protocols that try to reduce the amount of communication between the processors. (Lakovos, 2011) [4]

For such kind of hardware implementation, we can take the Intel Xeon (Nehalem) as a case of study, this processor requires a protocol called MESIF to maintain its cache coherence. We will not go through advanced details about this protocol, but what follows is a quick overview about its main functionality.

The word MESIF stands for Modified, Exclusive, Shared, Invalid, Forward and they corresponds to the states of a cache line:

- **Modified:** The cache line is present only in the current cache, and is dirty, it has been modified from the value in main memory. The cache is required to write the data back to main memory at some time in the future, before permitting any other read of the (no longer valid) main memory state. The write-back changes the line to the Exclusive state.
- **Exclusive:** The cache line is present only in the current cache, but is clean, it matches main memory. It may be changed to the Shared state at any time, in response to a read request. Alternatively, it may be changed to the Modified state when writing to it.
- **Shared:** Indicates that this cache line may be stored in other caches of the machine and is “clean” it matches the main memory. The line may be discarded (changed to the Invalid state) at any time.
- **Invalid:** Indicates that this cache line is invalid.
- **Forward:** Indicates that a cache should act as a designated responded for any requests for the given line. The protocol ensures that, if any cache holds a line in the Shared state, exactly one (other) cache holds it in the Forward state. The Forward state is a specialized form of the Shared state.

The approach to the Nehalem architecture is more modular than the core architecture which makes it much more flexible and customizable to the application. The architecture only consists of a few basic building blocks. The main blocks are a microprocessor core (with its own L2 cache), a shared L3 cache, a Quick Path Interconnect (QPI) bus controller, an Integrated Memory Controller (IMC), and graphics core. (Trent, 2009) [5]

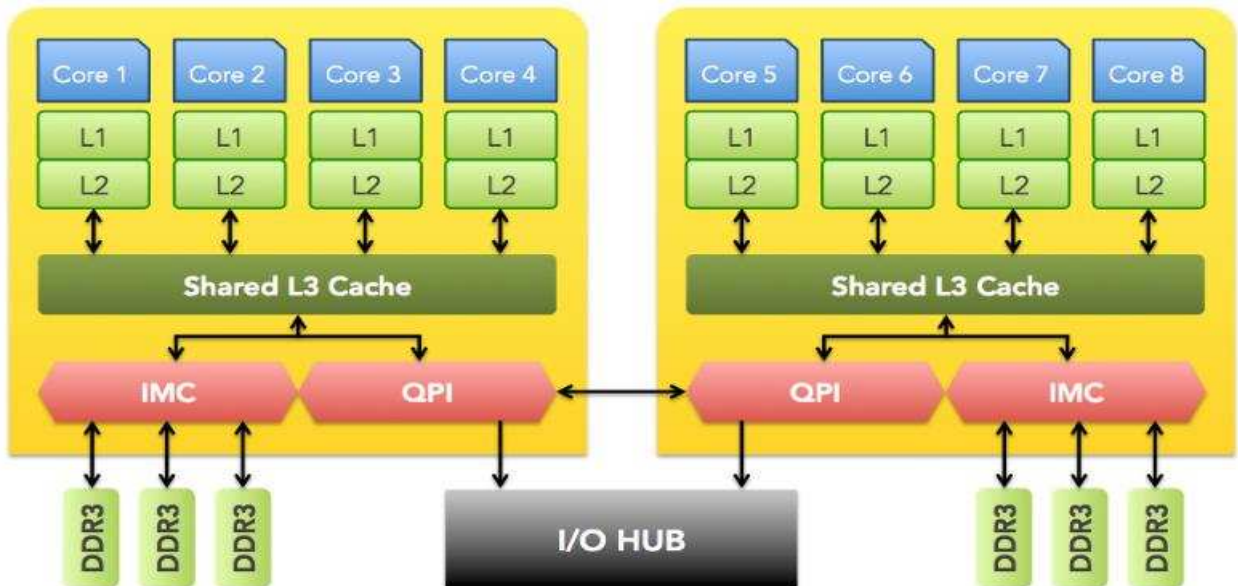


Figure 6.1: Eight core Multi-socket Nehalem Processor

2. 4. Interconnections

Many computer architectures are now adopting NUMA, this wouldn't be possible if not the advanced interconnection network technologies that allow faster data transfer. Three main types of interconnection fabrics can be distinguished: (Jean Loup, 2010)

- **Shared Bus:** Is the simplest form of interconnection is the shared bus. It has several advantages in addition to its simplicity such low cost, ease of use, and the possibility of simple cache coherence protocols because of the availability to broadcast information.
- **Crossbars:** They provide maximum concurrency between processors and the memory, they are used extensively in specialized embedded processors such as network processors where they provide the switch fabric for incoming and outgoing packets on different communications. Moreover, capabilities for buffering and priority scheduling are included in the design. Some large multiprocessors have used crossbars, but quite often in a hierarchical way.
- **Meshes:** Used in direct connection networks, each processor is at the same distance from each memory or from any other processor. In indirect interconnection networks processing elements are connected to other processing elements at various distances. Quite often, these interconnection networks are differentiated by their dimension.

Interactions between the processors for the purpose of coordination and synchronization, or exchange of data are obtained by communication through Message Passing Interface (MPI) over the interconnection network. In multiprocessor systems, the main task of the interconnection network is to transfer a message from a specific processor to a specific destination. The message may contain data or a memory request. The destination may be another processor or a memory module, it is required to perform the message transfer correctly as fast as possible, even if several messages have to be transferred at the same time. (Thomas & Gudula, 2010)

2. 5. Remote Access Memory

In NUMA machine accessing memory depends on location of memory this mean accessing local memory is faster than accessing none local memory, Remote access memory fired simply because wonted data is not located into local memory by firing method read or to write or modifies data by firing write and modifies methods as request to other node memory. [6]

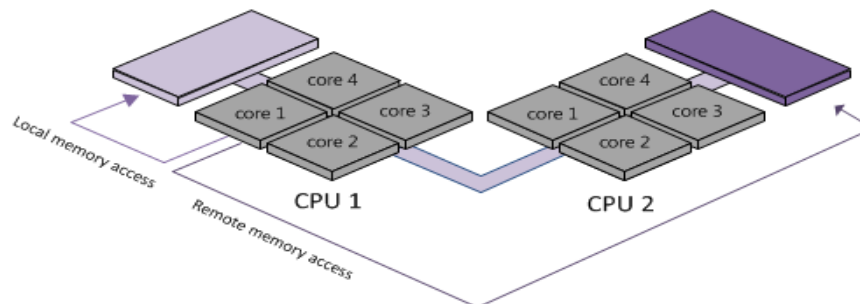


Figure 7.1: Remote access memory

3. Memory Hierarchy

A significant characteristic of the hardware development during the last decades has been the increasing gap between processor clock cycle time and main memory access time. The main memory is constructed based on Dynamic Random Access Memory (DRAM) which is way too slow compared to the CPU clock cycle, and so on the speed of the CPU is useless as it must wait for the data coming from the main memory. To use processor cycles efficiently, a memory hierarchy is typically used, consisting of multiple levels of memories with different sizes and access speed, see Figure 1.4. Only the main memory on the top of the hierarchy is built from DRAM, the other levels use SRAM (Static Random Access Memory), and the resulting memories are often called caches. The goal in the design of a memory hierarchy is to be able to access a large percentage of the data from a fast memory, and only a small fraction of the data from the slow main memory, thus leading to a small average memory access time. For multiprocessor systems where each processor uses a separate local cache, we face the problem of keeping a consistent view of the shared memory between all processors. It must be ensured that a processor accessing a data element always accesses the most recently written data value, also in the case that another processor has written this value. This is also referred to as cache coherence problem and will be detailed later on this chapter. [3]

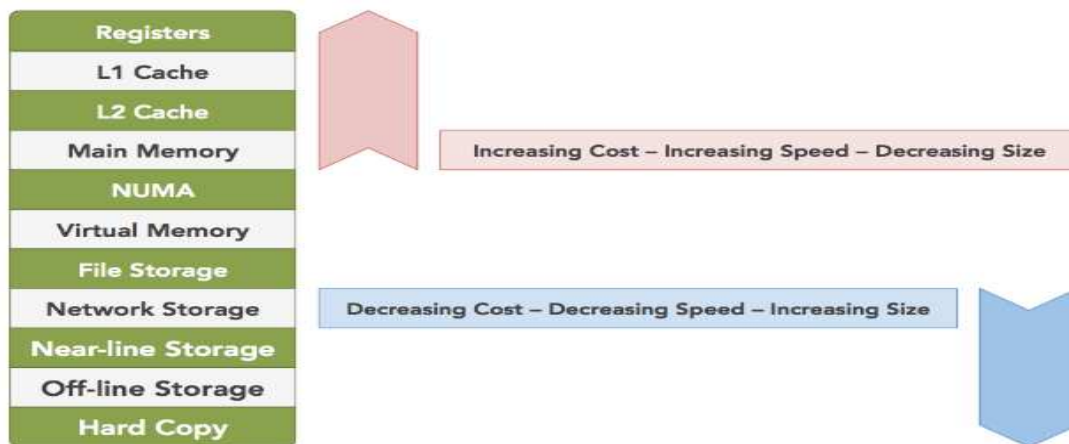


Figure 8.1: Memory Hierarchy

The design of a memory hierarchy may have a large influence on the execution time of parallel programs, and memory accesses should be ordered such that a given memory hierarchy is used as efficiently as possible. Moreover, techniques of parallel programming may also have an important influence to keep memory hierarchy more consistent. (Thomas & Gudula, 2010).

3. 1. Characteristics of Caches

A cache is a small, but fast memory between the processor and the main memory, it contains a copy of a subset of the data in main memory. Data is moved in blocks, containing a small number of words, between the cache and main memory as shown in Figure 1.5. These blocks of data are called cache blocks or cache lines. The size of the cache lines is fixed for a given architecture and cannot be changed during program execution. During program execution, the processor specifies memory addresses to be read or to be written as given by the load and store operations of the machine program.

The processor forwards the memory addresses to the memory system and waits until the corresponding values are returned or written. The processor specifies memory addresses independently of the organization of the memory system.

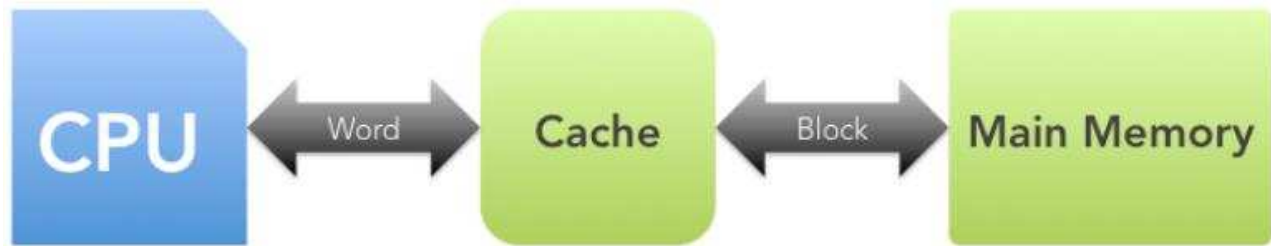


Figure 9.1: Data transport between cache and main memory

After having received a memory access request from the processor, the cache controller checks whether the memory address specified belongs to a cache line which is currently stored in the cache. If this is the case, a cache hit occurs, and the requested word is delivered to the processor from the cache. If the corresponding cache line is not stored in the cache, a cache miss occurs, and the cache line is first copied from main memory into the cache before the requested word is delivered to the processor. Since the access time to main memory is significantly larger than the access time to the cache, a cache miss leads to a delay of operand delivery to the processor. Therefore, it is desirable to reduce the number of cache misses as much as possible. What follows is a short overview of two important characteristics of caches.

3. 1. 1. Cache Size

Using the same hardware technology, the access time of a cache increases slightly with the size of the cache because of an increased complexity of the addressing. But using a larger cache leads to a smaller number of replacements as a smaller cache, since more cache blocks can be kept in the cache. The size of the caches is limited by the available chip area. Off-chip caches are rarely used to avoid the additional time penalty of off-chip accesses. Typical sizes for L1 caches lie between 8 and 128 Kilobytes memory words where a memory word is 4 to 8 Bytes long depending on the architecture. During the last years, the typical size of L1 caches has not increased significantly.

If a cache miss occurs when accessing a memory location, an entire cache block is brought into the cache. When designing memory hierarchy to fix the size of the cache blocks, it must be taken into consideration that using larger blocks reduces the number of blocks that fit in the cache when using the same cache size. Therefore, cache blocks tend to be replaced earlier when using larger blocks compared to smaller blocks. This suggests setting the cache block size as small as possible. Thus medium block sizes are used typically for L1 cache blocks.

3. 1. 2. Mapping of Memory Blocks

Data is transferred between main memory and cache in blocks of a fixed length. Because the cache is significantly smaller than the main memory, not all memory blocks can be stored in the cache at the same time. Therefore, a mapping algorithm must be used to define at which position in the

cache a memory block can be stored. The mapping algorithm used has a significant influence on the cache behavior and determines how a stored block is localized and retrieved from the cache. For the mapping, the notion of associativity plays an important role. Associativity determines at how many positions in the cache a memory block can be stored. The following methods are distinguished: (Thomas & Gudula, 2010)

- **Direct mapped cache:** each memory block can be stored at exactly one position in the cache.
- **Fully associative cache:** each memory block can be stored at an arbitrary position in the cache.
- **Set associative cache:** each memory block can be stored at a fixed number of positions.

Associativity is a term more likely to be discussed in more than just few lines, as there are many research articles that will not be detailed in this thesis. However, having an idea about it is favored as it is considered in mapping algorithms.

3. 2. Locality

The number of cache misses may have a significant influence on the resulting runtime of a program. If many memory accesses lead to cache misses, the processor may often have to wait for operands, and program execution may be quite slow. Since cache management is implemented in hardware, the programmer cannot directly specify which data should reside in the cache at which point in program execution. But the order of memory accesses in a program can have a large influence on the resulting runtime, and a reordering of the memory accesses may lead to a significant reduction of program execution time. In this context, the locality of memory accesses is often used as a characterization of the memory accesses of a program. There are common types of locality of reference used in computer architectures. Spatial and temporal locality can be distinguished as follows:

3. 2. 1. Spatial Locality

The memory accesses of a program have a high spatial locality, if the program often accesses memory locations with neighboring addresses at successive points in time during program execution. Thus, for programs with high spatial locality there is often the situation that after an access to a memory location, one or more memory locations of the same cache line are also accessed shortly afterward. In such situations, after loading a cache block, several of the following memory locations can be loaded from this cache block, thus avoiding expensive cache misses. The use of cache blocks comprising several memory words is based on the assumption that most programs exhibit spatial locality, for example when loading a cache block not only one but also several memory words of the cache block are accessed before the cache block is replaced again

3. 2. 2. Temporal Locality

The memory accesses of a program have a high temporal locality, if it often happens that the same memory location is accessed multiple times at successive points in time during program execution. Thus, for programs with a high temporal locality there is often the situation that after loading a cache block in the cache, the memory words of the cache block are accessed multiple times before the cache block is replaced again.

For programs with small spatial locality there is often the situation that after loading a cache block, only one of the memory words contained is accessed before the cache block is replaced again by another cache block. As for temporal locality, the situation is that after loading a cache block because of a memory access, the corresponding memory location is accessed only once before the cache block is replaced again. (Allen & Kennedy, 2002) [7]

3. 3. Shared Memory

When referring to a multitude of processors collaborating on the execution of a parallel program we often refer to an architecture that includes a shared memory. However, managing the information between processor cores can lead to some issues involving Symmetric Multiprocessing and Cache Coherence.

3. 3. 1. Symmetric Multiprocessing

A system that implements SMP has a single shared memory that is accessible by all processors in the system. The time of access to the memory is uniform, considering that the interconnections between main memory locations and each processor are equivalent, in most times the distance is taken as a reference. These interconnections provide communication for a small number of processors via a central bus as shown in Figure 1.6. This bus may get saturated if multiple processors are trying to access the shared memory simultaneously, this leads to an interconnect bottleneck as the processors have to wait for the memory to be ready before they can continue working.

The hierarchical multi-core design mentioned above seems to be a typical solution to solve this kind of problem depending on cache level. But as long as this hierarchy also includes shared memory such as L2 cache, we can deduce that each processor with more than two cores can be considered as an SMP system. [8]

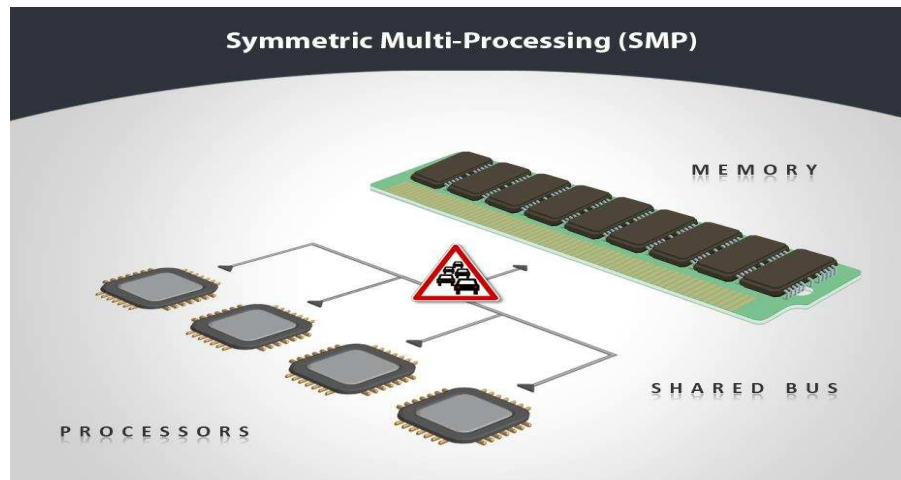


Figure 10.1: A Simple Symmetric Multiprocessing System

3. 3. 2. Cache Coherence

Shared memory can lead to a problem of coherence that can be defined when various cores of the processor copy of the same information from the shared memory, and as result various copies of the data will be stored in the cache of each core. Therefore, each copy of data can have a distinct value from other copies after it is being processed separately, it is then called incoherent or inconsistent.

The coherence of information between caches is a fundamental element since it ensures that a processor has access to the last value that was written. To maintain a certain level of coherency, we distinguish two kinds of algorithms depending on the architecture of the parallel system:

- **Cache-Memory Coherence:** Used in a single cache system, coherence between memory and the cache is maintained using one of two policies, write-through, and write-back. In write-through, the memory is updated every time the cache is updated, while in write-back, the memory is updated only when the block in the cache is being replaced. (Hesham & Mostafa, 2005)
- **Cache-Cache Coherence:** Used in in multiprocessing system, there are two fundamental policies to maintain coherence between caches, write-invalidate, and write-update. Write-invalidate maintains consistency by reading from local caches until a write occurs. Write-update maintains consistency by immediately updating all copies in all caches. All dirty bits are set during each write operation. After all copies have been updated, all dirty bits are cleared. (Hesham & Mostafa, 2005)

If the problem of cache coherence is allowed to occur, the resulting of the execution of the parallel program will be false and more likely leading to system failure. Thus, the system must handle this situation by updating the value of all data copies, such mechanism in the system is called coherency protocol.

The implementation of coherency protocols differs depending on the architecture of the execution platform, and so many protocols can be identified including, snooping protocols and directory-based protocols. [9]

- **Snooping Protocol**

This protocol relies on the property that all memory accesses are performed via the interconnections that can be used as broadcast medium. Thus, all entries can be observed by the cache controllers of each processor. When the cache controller observes a write into a memory location that is currently held in the local cache, it updates the value in the cache by copying the new value from the bus. Thus, the local caches always contain the most recently written values of memory locations. These protocols are also called update-based protocols since the cache controllers directly perform an update. There are also invalidation-based protocols in which the cache block corresponding to a memory block is invalidated so that the next read access must perform an update from main memory first. (Tanenbaum, 2001)

The technique of snooping relies on the use of a write-through policy and the existence of a broadcast medium, so that each cache controller can observe all write accesses to perform updates or invalidations to memory blocks. Each block has a state associated with it, and it may change as a result of the operations. These states are: valid, which means the copy is consistent with the global memory, and invalid, which means the copy is not consistent.

- **Directory-based Protocols**

This protocol is an alternative for snooping protocol, instead of using a broadcast medium it uses a central directory to store the state of memory blocks that may be held in cache. Instead of observing a shared broadcast medium, a cache controller can get the state of a memory block by a lookup in the directory. The directory can be held shared, but it could also be distributed among different processors to avoid access contention. (Culler & Singh, 1999)

At this section we have taken cache coherence as an important factor of performance increase for multi-core processors, and we have mentioned two most used protocols to maintain the coherence in cache memory. In this work these protocols (mostly snooping protocol) are to be considered function and they do not constitute the aiming of this research.

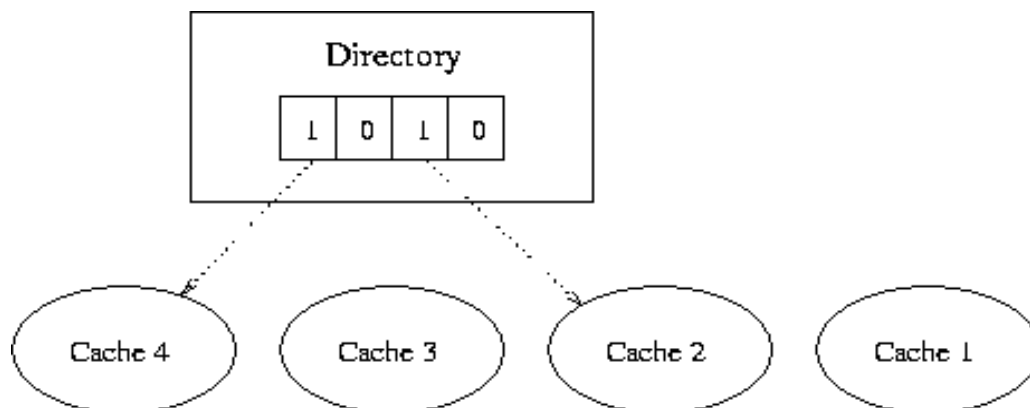


Figure 11.1: Directory base Protocol Schema

Typically 3 processors involved

- Local node where a request originates (interact with **CPU cache**)
- Home node where the memory location of an address resides (interact with **directory** in memory)
- Remote node has a copy of a cache block, whether exclusive or shared (interact with **CPU cache**)

4. Summary

Multi-core computers have been introduced to the world in the era of increasing need for higher performance, this need has led to a new computing paradigm. In this chapter we took a closer look at the most recent multi-core architecture, and we have learned that a bigger number of core doesn't mean necessary a better performance. CPUs are strongly related to every component in the computer inner system, and because memory is in direct interaction with the CPU, it is also acknowledged that improving the performance means a better memory management and less access time. We have also learned that locality and coherency of the memory are important factors to consider later on for the purpose of scheduling.

Also in this chapter we have addressed a particular case of computer architecture that is NUMA multi-core system. This last is our point of interest in this work as we have gone through some details of such architecture and some recent implementations of it in nowadays computing systems. It is clear that none of multi-core or multiprocessors possibilities would exist if not the advanced interconnection networks, as they are responsible for every transaction between the two main components in our research, which are the CPU and the memory. Data transfer between these two components is an important factor of performance, and for this reason we have suggested to give a hint about interconnections, but they remain hardware elements that the scheduler has no policy over it, but it also should be considered.

This chapter represents a general overview of multi-core architectures and NUMA systems in a manner of understanding the hardware structure of the platform we attend to work on by building and testing a scheduler that fits it. In the next chapter we will go through some details of the existing simulators that target NUMA systems and discuss some related works.

1. Simulation

Simulation is the imitation of the operation of a real-world process or system over time. The act of simulating something first requires that a model be developed; this model represents the key characteristics or behaviors/functions of the selected physical or abstract system or process. The model represents the system itself, whereas the simulation represents the operation of the system over time.

1. 1. Analytic simulation

Analytical models are mathematical models that have a closed form solution. Or in other words the solution to the equations used to describe changes in a system can be expressed as a mathematical analytic function.

Analytical models were more commonly used up until the 1990s before numerical modeling became possible with the advance in the capabilities and speed of computers. They were used to verify assumptions on the heat and fluid transport in the geothermal system. Currently, analytical modeling is rarely used due to the complexity of the geothermal system.

In any system (or model), there are parameters and other conditions (e.g. initial conditions) with make results different. In simulation, values of these effective parameters must be specified and result will depends just on that value of parameter. But analytical result, give a general description of system for any value of parameters. For example, suppose that we want to solve a simple linear $dx/dt=a*x$, which is parameter. In numerical result, we choose an initial condition and a value for "a" and find solution, but we can't deduce general result for any initial condition and any value of "a", while analytical result (if can be derived) gives result for any initial condition and any value of "a".

1. 2. Discrete event simulation

In the field of simulation, a discrete-event simulation (DES), models the operation of a system as a discrete sequence of events in time. Each event occurs at a particular instant in time and marks a change of state in the system. Between consecutive events, no change in the system is assumed to occur. Thus the simulation can directly jump in time from one event to the next.

This contrasts with continuous simulation in which the simulation continuously tracks the system dynamics over time. Instead of being event-based, this is called an activity-based simulation. Time is broken up into small time slices and the system state is updated according to the set of activities happening in the time slice. Because discrete-event simulations do not have to simulate every time slice, they can typically run much faster than the corresponding continuous simulation.

As an example of discrete-event simulation, consider the logic circuit presented:

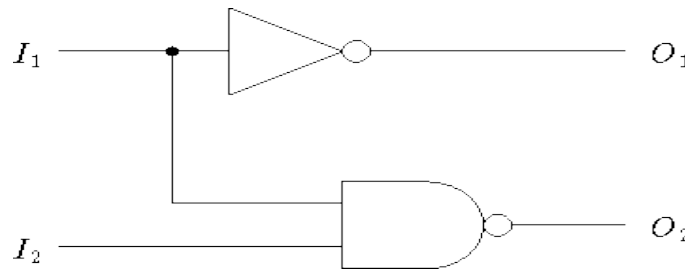


Figure 12.2: a simple analytic module

An effective DES process must include, at a minimum, the following characteristics:

- * Predetermined starting and ending points, which can be discrete events or instants in time.
- * A method of keeping track of the time that has elapsed since the process began.
- * A list of discrete events that have occurred since the process began.
- * A list of discrete events pending or expected (if such events are known) until the process is expected to end.
- * A graphical, statistical, or tabular record of the function for which DES is currently engaged.

DES is commonly used to monitor and predict the behavior of investments. The stock market is a classic example. DES can also help administrators predict how a network will behave under extraordinary conditions, such as the Internet during a major disaster.

2. Multi-core & NUMA simulator benefits

As mentioned above there are many different aspects about Multi-core and NUMA but the most important is, what are the benefits after creating a simulator for those systems?

Architectural simulators are very useful for the following purposes:

- Evaluating different hardware designs without building costly physical hardware systems.
- Enabling the opportunities to access non-existing computer components or systems obtaining detailed performance metrics: A single execution of simulators can often generate a large set of performance data.
- Debugging: Debugging on real hardware typically requires re-booting and re-running the code to reproduce the problems. In contrast, some simulators have a fully controlled environment and allow software developers to run code backward once an error is detected.

Full system simulation can speed the system development process by making it easier to detect, recreate and repair flaws. The use of multi-core processors is driving the need for full system

simulation, because it can be extremely difficult and time consuming to recreate and debug errors without the controlled environment provided by virtual hardware. This also allows the software development to take place before the hardware is ready, thus helping to validate design decisions.

3. Simulation

Hardware designer: designers simulate real hardware system by construction.

Software designer: need to simulate the hardware virtually before create a real prototype and by performing different actions

3. 1. System:

System in simulation is a collection of entities that act and interact with each other toward the accomplishment of some logical end by exchanging messages and information.

3. 2. Model:

Model in simulation is abstract representation of system, example of class in oriented object programming. A representation of the system and study it as a surrogate for the actual system.

3. 3. Why Model?

- System is not physically exists System is not physically exists
- Building a system is expensive
- Measuring a system is time-consuming

3. 4. Characterizing a Model:

3. 4. 1. Static & Dynamic

Is time a significant variable?

3. 4. 2. Deterministic & Stochastic

Does the model contain stochastic components?

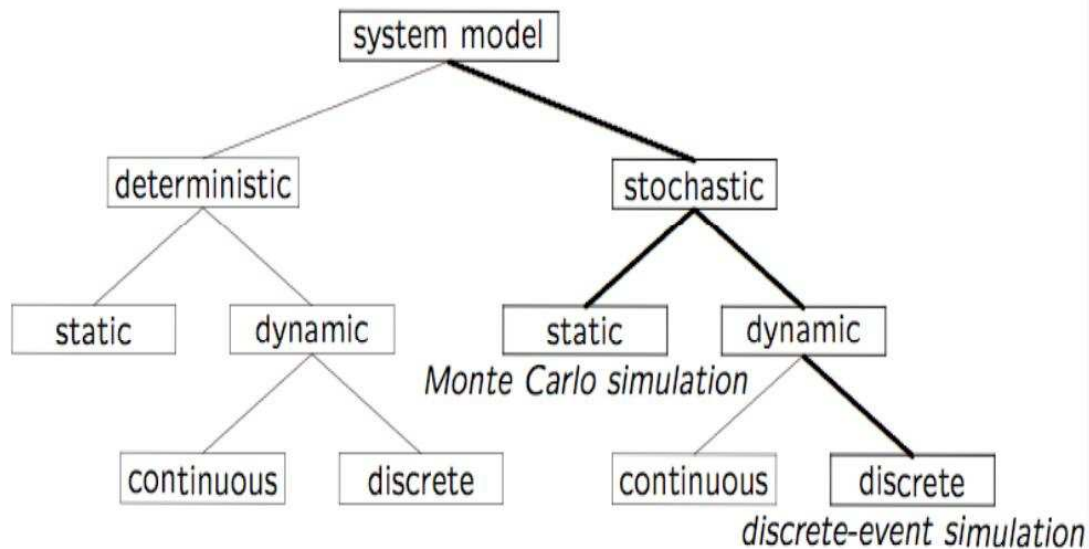
3. 4. 3. Continuous & Discrete

- Does the system state evolve continuously or only at discrete points in time?
- Continuous: classical mechanics
- Discrete: queuing, inventory, machine shop models.

Discrete system: State variables change instantaneously at separated point in time, e.g., a bank, since state variables - number of customers, change only when a customer arrives or when a customer finishes being served and departs.

Continuous system: State variable change continuously with respect to time, e.g., airplane moving through the air, since state variables -position and velocity change continuously with respect to time.

Figure 13.2: Model Taxonomy



3. 5. DES Discrete Event Simulation

Models a system whose state may change only at discrete point models a system whose state may change only at discrete point in time.

- **State**
- A collection of attributes or state variables that represent the entities of the system.
- **Event**
- An instantaneous occurrence in time that may alter the state of an instantaneous occurrence in time that may alter the state of the system
- An event initiates an activity, which is the length of time during which entities engage in some operations.
- Entities, attributes, events, activities and the interrelationships between these components are defined interrelationships between these components are defined in the model of the system

3. 6. Why simulators for NUMA?

- Test this on real NUMA hardware is very expensive on both time and money.
- Simulator is best solution for us. Where, designers could use a simulator to implement and test different approaches, to analyze it, improve performance and scalability before concrete realization.

“Simulation is the process of designing a model of a real system and conducting experiments with this model for the purpose either of understanding the behavior of the system or of evaluating various strategies (within the limits imposed by a criterion or set of criteria) for the operation of a system.”

-Robert E Shannon 1975

4. Current Simulators

Simulation plays an important role in the research of computer architecture. In the past years, a bunch of simulators have been developed, such as Gem5 ruby, MCoresim, graphite and recently SimNUMA.

4. 1. Graphite

Graphite is a new parallel, distributed simulator infrastructure designed to enable rapid high-level architectural evaluation and software development for future multi-core architectures. It provides both functional and performance modeling for cores, on-chip networks, and memory subsystems including cache hierarchies with full cache coherence. The design of Graphite is modular, allowing the different models to be easily replaced to simulate different architectures or trade off performance for accuracy. Graphite runs on commodity Linux machines and can execute unmodified p thread applications. Graphite will be released to the community as open-source software to foster research and software development for future multi-core processors. [11]

A variety of techniques are used to deliver the performance and scalability needed to perform useful evaluations of large multi-cores including: direct execution, multi-machine distribution, analytical modeling and lax synchronization.

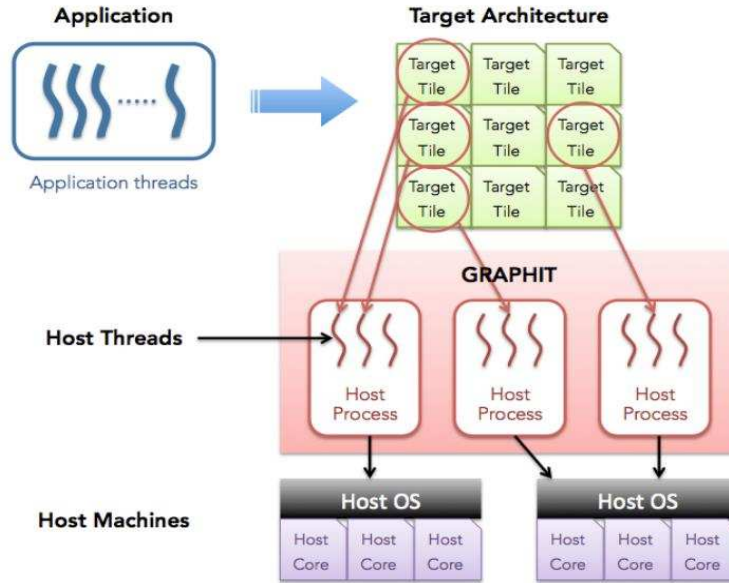


Figure 14.2: GRAPHIT Simulator Architecture

4. 2. MCoresim

The goal of MCoresim mapping is to map a set of processor-cores to a specified application in an optimized manner (software hardware mapping). In multi-core systems, the hardware topology is pre-configured, so MCoresim will have to map and schedule applications and their communications on the available hardware resources.

4. 3. Gem5 ruby

The goal of the gem5 simulator is to be a community tool focused on architectural modeling. Three key aspects of this goal are flexible modeling to appeal to a broad range of users, wide availability and utility to the community, and high level of developer interaction to foster collaboration. [10]

4. 4. simNUMA

Presented an execution-driven, full-system simulator dedicated for NUMA-architecture systems. The main characteristics of the SimNUMA include: it uses the same type of processor with the target machine in the host system to avoid complex and inefficient modeling and simulation of micro-architecture and instruction-set for commercial available processors. on the basis of that, a new method to capture and simulate remote-memory accesses efficiently is proposed. in addition, parallel simulation is supported to achieve scalability. Consequently, the simulation performance is improved

significantly. Moreover, modeling and simulation of different kinds of interconnection networks are supported in SimNUMA. [14]

5. Summary

The large scale of NUMA architecture is a complex system, where hardware is moving faster than software. The hardware design is so expensive in time and money.

In this chapter, we have discussed the important of simulator for such a complex system that will be our goal in this thesis. We have classified our simulator on various options that we have seen. Also, we have seen existing simulator that exist for multi-core and NUMA simulator by showing weak and strong points.

To take full advantage of NUMA architecture, we are next going to combine policies and test as possible strategies to come up with a perfect combination, and more understanding of policies and their behavior in a NUMA machine. This last is not available, therefore, using a simulator is a necessity for our research, we have done some search and discussed some of the existing simulators, but these simulators do not implement fully the NUMA architecture, most of them only stay under the context of UMA architecture. Also, they are not open source for computer sciences community. And so, we have decided to create our own NUMA machine simulator that gives us the possibility to run NUMA policies on and do our test. This is what we are going to see in the next chapter.

1) Introduction

Designing and modeling a simulator is a traditional term even for mono processors computers, but the more processors architectures get complex the more simulator extends its dimension and gets more complicated. At first stages, simulator of operating system had only two resources to design model, how Memory handle data? And how CPU loads task? This simple implementation of simulators has evolved by the introduction of parallel systems, and now many arguments are taken into account.

The complexity of parallel programming motivates research into automatic parallelization techniques and tools. One particularly difficult part of setting up a simulator is respecting software engineering roles and red lines. We have worked on this part a lot. In order, to create a reusable, documented, structured and organize code.

Dealing with NUMA multicore systems is a real challenge for modern operating systems, in fact, creating modeling a simulator in this situation has a totally different dimension. In this chapter we will present the aspects related to NUMA environment and arguments that should be considered by this simulator. We will present some of these challenging part, study their efficiency and how much they adapt with the current NUMA systems. But before that, we will start this chapter by introducing the general problem of designing a simulator for this system.

2) NUMA Machine Topology

In this section we will address the conception of our NUMA machine simulator, we will explain the main functionality of it and the procedures we have been thru while building it. One important and crucial procedure that took most of our time was in fact by asking this question: How is the simulator going to get NUMA machine topology and how is it going to create an object model that simulate it? Is it going to be based on a static or a dynamic topology? Of course, we could have built a simulator based on a static topology, let's say a NUMA machine with 2 NUMA Nodes, and run our tests on, and it could have saved us much of time, but what if someone else want to run a test on a NUMA machine with 4 NUMA Nodes. We didn't want this simulator to die by the end of this research, and therefore we have built a simulator that creates an object model based on dynamic NUMA topologies, and it is also Open Source, and so any developer can run test on multiple NUMA environments with the possibility of adding other functions and theories depending on their study.

In order to achieve that, we have based our simulator on the Portable Hardware Locality software. The output of this last is an XML file that represents exactly the hardware topology of the architecture where it applied. The following session is dedicated to help up understand more about topologies and hardware locality software.

2. 1) Hardware Topologies

2. 1. 1) Definition

Hardware topology is the pattern of computer elements. It can be physical, mapping hardware configuration, or logical, mapping the interactions between these elements. There are many identified topologies but they are not strict, which means that any of them can be combined. However, each topology has a different standard and may use different hardware methods so they are not interchangeable. (Cory, 2011)

2. 1. 2) NUMA Topology

NUMA systems, independent of processor architecture and operating system, use some numbering scheme that assigns an ID or index (or set of indexes) to each hardware thread, core and NUMA locality domain for identification. System tools such as libraries for affinity control use these indexes, so it is important to know about the scheme. For convenience, there is sometimes an abstraction layer, which allows the user to specify entities like NUMA nodes, sockets, cache groups, processor cores and so on. (Georg & Gerhard, 2011)

Either NUMA or any other architectures topology can be represented on many forms, such as a structures file or a graph as long as it provides the necessary information to do the abstraction of the object model, the following figures represents a simple NUMA topology.

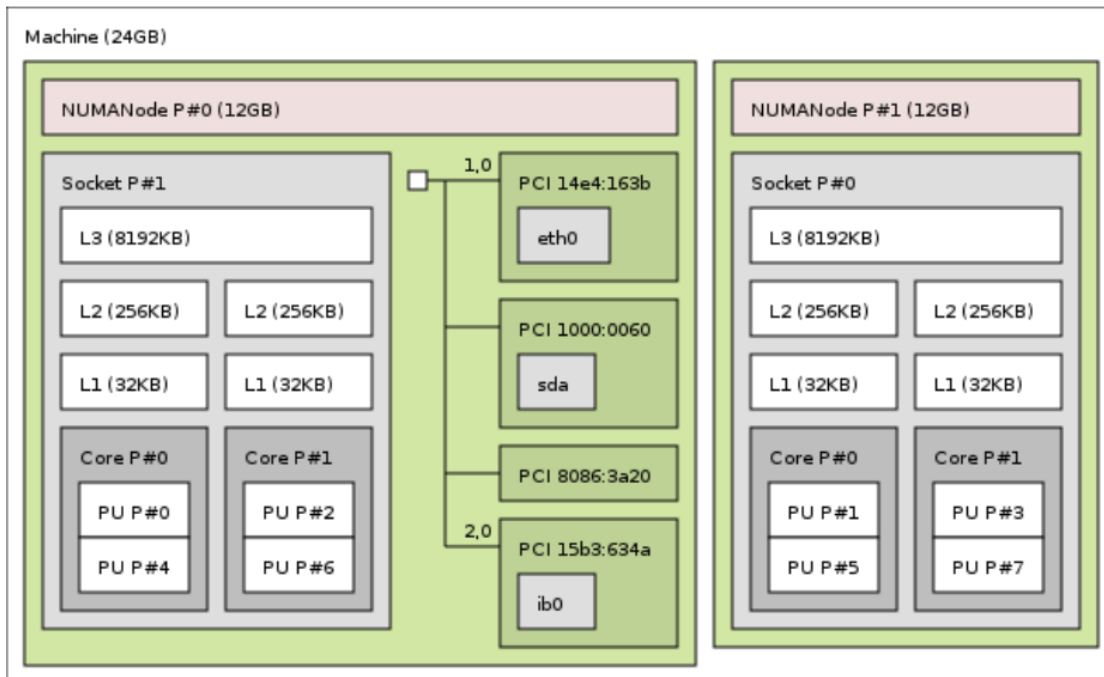


Figure 15.3: A simple NUMA architecture topology

As mentioned before, each entity that belongs to a NUMA system must be identified, in Figure 4.1 there is a clear hierarchical organization starts with the machine and then NUMANodes identified by #0 and #1 and then going deeper to the sub elements, with each one identified. This identification is what our simulator is going to rely on while targeting children or parent of specific object.

2. 2) HwLoc

The Portable Hardware Locality (hwloc) software package provides a portable abstraction (Versions, architectures, etc.) Of the hierarchical topology of modern architectures including NUMA memory nodes, sockets, shared caches, cores and simultaneous multithreading. It also gathers various system attributes such as cache and memory information as well as the locality of I/O devices such as network interfaces, InfiniBand HCAs or GPUs. It primarily aims at helping applications with gathering information about modern computing hardware so as to exploit it accordingly and efficiently. [13]

There are many possible implementation of HwLoc but in our case we only used the output of its execution on real NUMA machines. This output is the topology of these machines structured in an XML file. This last, is the first thing that the simulator requires before generating data and tasks or executing simulation process. Figure 4.2 represents the content of NUMA topology XML file generated by HwLoc, the architecture represented in this file is a machine with 2 NUMA nodes with a socket and a bridge (interconnection) for each. There are also other sub elements such as caches and cores that we will talk about later in this chapter.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE topology SYSTEM "hwloc.dtd">
3 <topology>
4   <object type="Machine" os_index="0" cpuset="0x00ffffff" complete_cpuset="0x00ffffff"
5     <page_type size="4096" count="0"/>
6     <page_type size="2097152" count="0"/>
7     <info name="DMIProductName" value="ProLiant SL390s G7"/>
8     <info name="DMIProductVersion" value=""/>
9     <info name="DMIBoardVendor" value="HP"/>
10    <info name="DMIBoardName" value=""/>
11    <info name="DMIBoardVersion" value=""/>
12    <info name="DMIBoardAssetTag" value=""/>
13    <info name="DMIChassisVendor" value="HP"/>
14    <info name="DMIChassisType" value="25"/>
15    <info name="DMIChassisVersion" value=""/>
16    <info name="DMIChassisAssetTag" value=""/>
17    <info name="DMIBIOSVendor" value="HP"/>
18    <info name="DMIBIOSVersion" value="P69"/>
19    <info name="DMIBIOSDate" value="10/21/2010"/>
20    <info name="DMISysVendor" value="HP"/>
21    <info name="Backend" value="Linux"/>
22    <info name="OSName" value="Linux"/>
23    <info name="OSRelease" value="2.6.27.39-0.3-perfctr"/>
24    <info name="OSVersion" value="#1 SMP 2009-11-23 12:57:38 +0100"/>
25    <info name="HostName" value="mirage004"/>
26    <info name="Architecture" value="x86_64"/>
27    <distances nbobjs="2" relative_depth="1" latency_base="10.000000">
28      <latency value="1.000000"/>
29      <latency value="2.000000"/>
30      <latency value="2.000000"/>
31      <latency value="1.000000"/>
32    </distances>
33    <object type="NUMANode" os_index="0" cpuset="0x00555555" complete_cpuset="0x00555555"
34      <page_type size="4096" count="4715975"/>
35      <page_type size="2097152" count="0"/>
36      <object type="Socket" os_index="0" cpuset="0x00555555" complete_cpuset="0x00555555"
37        <object type="Bridge" os_index="0" bridge_type="0-1" depth="0" bridge_pci="0000:[00
146      </object>
147      <object type="NUMANode" os_index="1" cpuset="0x00aaaaaa" complete_cpuset="0x00aaaaaa"
148        <page_type size="4096" count="4718591"/>
149        <page_type size="2097152" count="0"/>
150        <object type="Socket" os_index="1" cpuset="0x00aaaaaa" complete_cpuset="0x00aaaaaa"
202        <object type="Bridge" os_index="1" bridge_type="0-1" depth="0" bridge_pci="0000:[10
218      </object>

```

Figure 16.3: NUMA topology generated by HwLoc

Going back to topology definition and the identification of the system elements, it is obviously that HwLoc uses the index given by the OS as an identifier. We can use that to identify each attribute of the machine by sending the elements IDs to `os_index` attribute.

2. 2. 1) DTD of XML file

DTD: Document Type Definition defines the structure and the legal elements and attributes of an XML document. [13]

On our xml file we have this DTD file from the project website

```

<!-- Copyright © 2009      CNRS
Copyright © 2009-2015 Inria. All rights reserved.

```

```
Copyright © 2009-2011 Université Bordeaux.  
See COPYING in top-level directory.  
-->  
<!ELEMENT topology (object)+>  
<!ELEMENT root (object)+>  
<!ELEMENT object (page_type*,info*,distances*,userdata*,object*)>  
<!ATTLIST object type (System | Machine | Misc | Group | NUMANode | Package | Cache  
| Core | PU | Bridge | PCIDev | OSDev) #REQUIRED>  
<!ATTLIST object os_level CDATA "-1" >  
<!ATTLIST object os_index CDATA "-1" >  
<!ATTLIST object name CDATA "" >  
<!ATTLIST object local_memory CDATA "0" >  
<!ATTLIST object cache_size CDATA "0" >  
<!ATTLIST object cache_linesize CDATA "0" >  
<!ATTLIST object cache_associativity CDATA "0" >  
<!ATTLIST object cache_type CDATA "0" >  
<!ATTLIST object huge_page_size_kB CDATA "0" >  
<!ATTLIST object huge_page_free CDATA "0" >  
<!ATTLIST object depth CDATA "-1" >  
<!ATTLIST object cpuset CDATA "0" >  
<!ATTLIST object complete_cpuset CDATA "" >  
<!ATTLIST object allowed_cpuset CDATA "" >  
<!ATTLIST object nodeset CDATA "" >  
<!ATTLIST object complete_nodeset CDATA "" >  
<!ATTLIST object allowed_nodeset CDATA "" >  
<!ATTLIST object bridge_type CDATA "" >  
<!ATTLIST object bridge_pci CDATA "" >  
<!ATTLIST object pci_busid CDATA "" >  
<!ATTLIST object pci_type CDATA "" >  
<!ATTLIST object pci_link_speed CDATA "0." >  
<!ATTLIST object osdev_type CDATA "" >  
<!ELEMENT page_type EMPTY>  
<!ATTLIST page_type size CDATA #REQUIRED>  
<!ATTLIST page_type count CDATA #REQUIRED>  
<!ELEMENT info EMPTY>  
<!ATTLIST info name CDATA #REQUIRED>  
<!ATTLIST info value CDATA #REQUIRED>  
<!ELEMENT distances (latency*)>  
<!ATTLIST distances nbobjs CDATA #REQUIRED>  
<!ATTLIST distances relative_depth CDATA #REQUIRED>  
<!ATTLIST distances latency_base CDATA #REQUIRED>  
<!ELEMENT latency EMPTY>  
<!ATTLIST latency value CDATA #REQUIRED>  
<!ELEMENT userdata (#PCDATA)>  
<!ATTLIST userdata name CDATA "" >  
<!ATTLIST userdata length CDATA "0" >
```

```

<!ATTLIST userdata encoding CDATA "" >
<!ELEMENT topologydiff (diff)*>
<!ATTLIST topologydiff refname CDATA "">
<!ELEMENT diff EMPTY>
<!ATTLIST diff type CDATA #REQUIRED>
<!ATTLIST diff obj_depth CDATA "-1" >
<!ATTLIST diff obj_index CDATA "-1" >
<!ATTLIST diff obj_attr_type CDATA "-1" >
<!ATTLIST diff obj_attr_index CDATA "-1" >
<!ATTLIST diff obj_attr_name CDATA "" >
<!ATTLIST diff obj_attr_newvalue CDATA "" >
<!ATTLIST diff obj_attr_oldvalue CDATA "" >

```

3) High Level NUMA Simulator

3. 1) Why high level NUMA simulator?

We choose this type of simulators for the demands by the computer science community and also for so many reasons of usability and accessibility. Moreover, it is easier to understand. It is more like human language and less like machine language. Also, it's portable and can work across different CPU families, not like other existing NUMA simulator such as simNUMA [2] where it's related to kernel.

3. 2) What are the need of High level NUMA simulator

As munched above on section why high level NUMA simulator, the benefit of creating a high level one are the same reason of reason we create this first place. But the main reason of all reasons is portability, imagine one library can be used on cross platforms and also documented and structured.

3. 3) Structured of our simulator:

Relying on HwLoc [15] we could build a simulator that creates an object model based on the topology informations provided by XML file gathered by thids tools from a real NUMA platform. The topology, in other words, we managed to do a high level emulation by doing a reverse engineering of jHwLoc which is the JAVA version of HwLoc, and because they are exactly the same, the generated XML is identical, and can be parsed by the simulator. But before addressing the engineering session, let's have an overview about the design of our emulation software.

3. 4) Simulator Design

3. 4. 1) Separation between model and simulator

We have done that in purpose because in computer science, a **library** is a collection of implementations of behavior, written in terms of a language, which has a well-defined interface by which the behavior is invoked. This means that as long as a higher level program uses a library to make system calls, it does not need to be re-written to implement those system calls over and over again. In addition, the behavior is provided for reuse by multiple independent programs. A program invokes the library-provided behavior via a mechanism of the language. For example, in a simple imperative language such as C, the behavior in a library is invoked by using C's normal

function-call. What distinguishes the call as being to a library, versus being to another function in the same program, it's the way that the code is organized in the system.

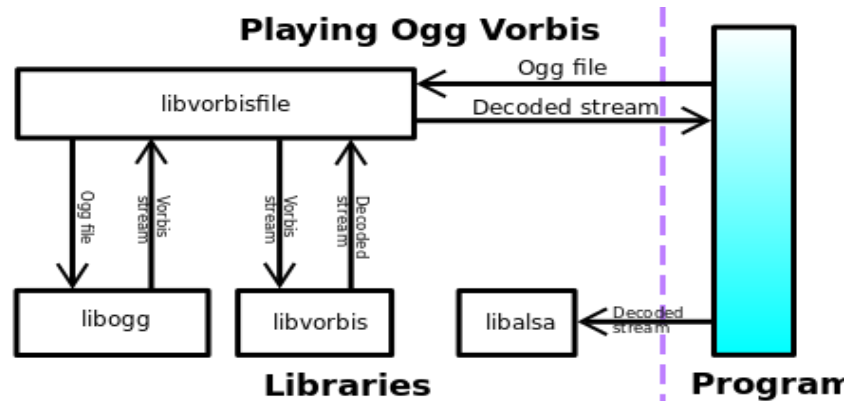


Figure 17.3: Illustration of an application which uses libvorbisfile

“The value of a library is the reuse of the behavior. When a program invokes a library, it gains the behavior implemented inside that library without having to implement that behavior itself. Libraries encourage the sharing of code in a modular fashion, and ease the distribution of the code.”

3. 4. 2) Static libraries

When linking is performed during the creation of an executable or another object file, it is known as static linking or early binding. In this case, the linking is usually done by a linker, but may also be done by the compiler. A static library, also known as an archive, is one intended to be statically linked. Originally, only static libraries existed. Static linking must be performed when any modules are recompiled.

3. 4. 3) Shared libraries

A **shared library** or **shared object** is a file that is intended to be shared by executable files and further shared objects files. Modules used by a program are loaded from individual shared objects into memory at load time or run time, rather than being copied by a linker when it creates a single monolithic executable file for the program.

Shared libraries can be statically linked, meaning that references to the library modules are resolved and the modules are allocated memory when the executable file is created. But often linking of shared libraries is postponed until they are loaded

3. 5) Design Approach:

In this section, we will address the design pattern of our simulator, discussing the structure of it, and the most important interactions between its modules, we are talking here about the structured of the program, that is responsible for building the simulator from and XML file. The resources interfaces. That will generate an objects from classes to be represent the functions that resources need to do in normal physical NUMA machine, and the hierarchical executer. Which will apply a set of NUMA functions on the generated objects within the emulated platform.

To achieve this, we are going to use UML annotations. With the implementation of OOP with Java. We have integrated many Java classes to build the simulator that will represent these classes with some Diagrams that will be as follow: Package Diagram then Class Diagrams with more details and information. Also, we will represent the Sequence Diagram to understand the interactions between the main operators in a measure of time relay. Finally, we will conclude with Activity Diagram to understand the scenario and process of the creating of model.

3. 6) Design and modeling process:

Computer simulation modeling is a discipline gaining popularity in both government and industry. Computer simulation modeling can assist in the design, creation, and evaluation of complex systems. Designers, program managers, analysts, and engineers use computer simulation modeling to understand and evaluate ‘what if’ case scenarios. That why we abstract and inspired from real and existing machine architectures of NUMA machine.

3. 7) How we design the NUMA model:

We took a look at in first time on basic machine with one CPU with one core and one memory bank. This search allow us to defined the basic methods such as read() and write() on memory resource. Also, we defined load(), store() and Compute () methods for CPU resource. Then, we take it little bit further where, we study multiple cores in single CPU. We have found new resources such as bus and cache. In this point we have developed a hierarchical model. Where CPU is the father of all buses and each buses has cores and cache as children. Moreover, we kept looking on more complex architecture. We have faces new entity like InterConnectionNetwork (ICN) and new concept called Node. As it munched above, developing a hierarchical model all us to add those last two as children or as father position. With receive() and send() for ICN.

The physical topology of NUMA platform that has a hierarchical nature inspired us to develop and model out model approach. The following entities are required and has the most important work.

Modeling Entities:

ICN: This entity is the most valuable component that related all nodes on NUMA machine and handle all the communication process such as Remote memory access.

- Attributes:
 - Bandwidth
 - Latency
- Methods:
 - Send();
 - Receive();

Node: This entity gathers the big part of machine. Where we can find Bus, CPU, Core and Cache resources.

- Attributes:
 - ID
 - type
- Methods:
 - getMemory();
 - getCPUCount();

RAM: This entity is the bank memory of our simulator, with basic functionalities.

- Attributes:
 - Size
 - PageSize
 - WordSize
- Methods:
 - Read();
 - Write();

Core: This entity handle all calculation and tasks process.

- Attributes:
 - speedUnit
- Methods:
 - Load();
 - Store();
 - Compute();

Cache: This entity can play the role of RAM but it has more functions and less storage.

- Attributes:
 - Size
 - LineSize
- Methods:
 - Read();
 - Write();

4) Model hypotheses:

Our model will be focusing more about communication and computation aspects, all others aspects will be ignored and also we will are not handling the error. But note that we hypotheses that our communication resource will not take in consideration the routing problems, all channels (ICN, Bus) are homogeny and reliable, bank memory storage functions time in read() and write() and other communication, computation functions such as process(), send() and receive() will be uniform.

- Channel communication

$T_{pd} = \text{constant for all cores}$

- Channel communication

$T_{pd} = \text{constant for all nodes}$

T_{pd} is the propagation delay time for one unit data on the interconnection network from source to reach destination

- In local node

$T_{LA} = \text{constant for all cores}$

T_{LA} : is the local access memory time for all cores to local memory for one unit of data.

- Remote memory access

$T_{RA} = \text{constant for all nodes}$

T_{RA} : is the remote access memory access time for all nodes to no local memory for one unit of data.

- Executions time all instruction have the same running time on all cores

$T_i = \text{constant for all cores}$

T_i : is the time of executing an instruction on core.

Finally, all sockets are homogeny and has the same architecture and structured.

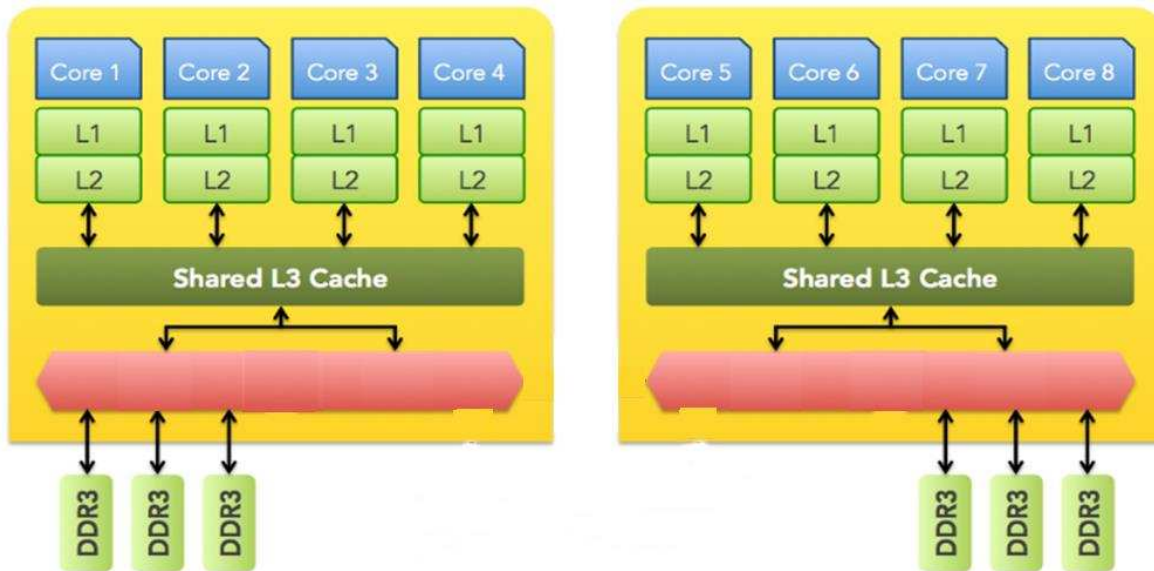


Figure 18.3: Homogeneity of sockets

5) Class Diagram

At this point we will talk about the most important and used classes, these are either updated from jHwLoc or created by us. These classes are encapsulated under packages to maintain simplicity and usability, therefore, we will start by the package diagram.

Package Diagram

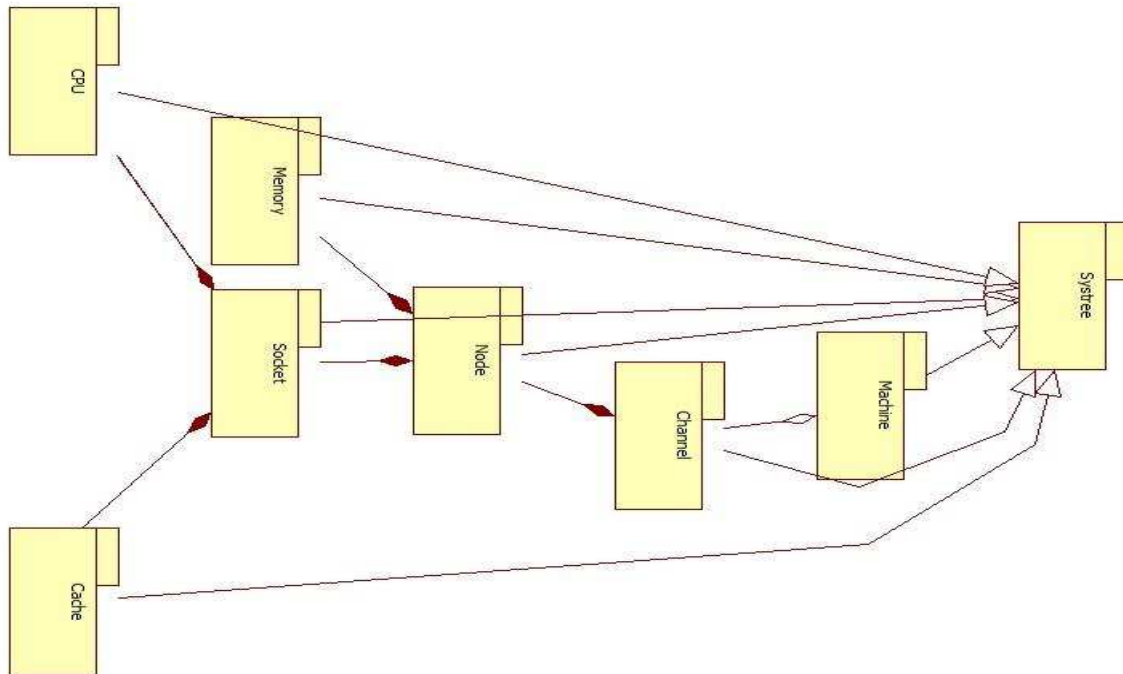


Figure 19.3: Package Diagram modeling

First look at this diagram, we can deduce that SysTree package has most classes and most relations, this is because the NUMA interface resources is extended from this package. It contains the hierarchical interface that structure components. Other packages like Machine and ICN are responsible for creating NUMA resources and generate it functionalities. We will go through more details of these objects including their methods later on.

Class Diagram

Regardless to the belonging of classes to packages class diagram represents the classes and their relationships in order to construct our solution. It consist of creating a close object model to a real NUMA machine, by creating an object that simulate each part of the system, such as cores, memories and interconnections, the class diagram would look like the following.

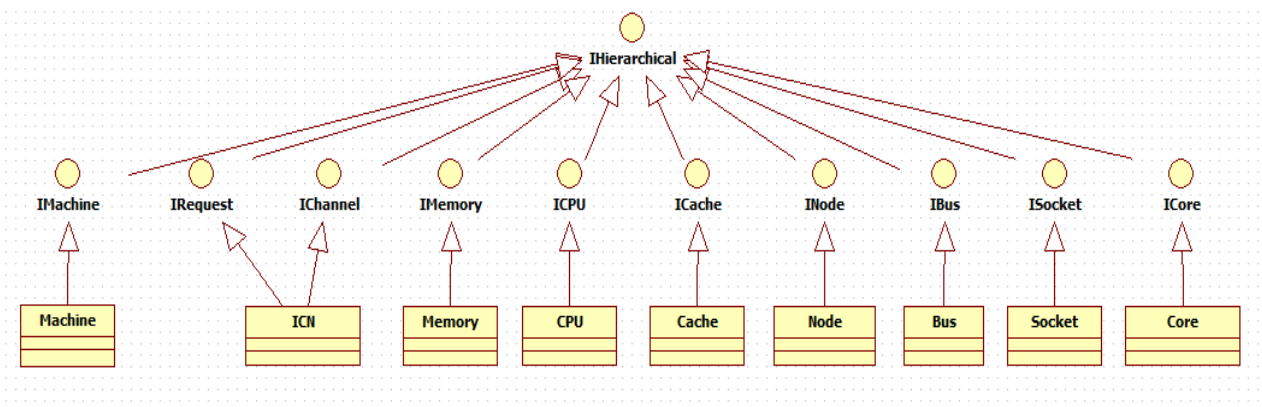


Figure 20.3: Interface Structure Diagram

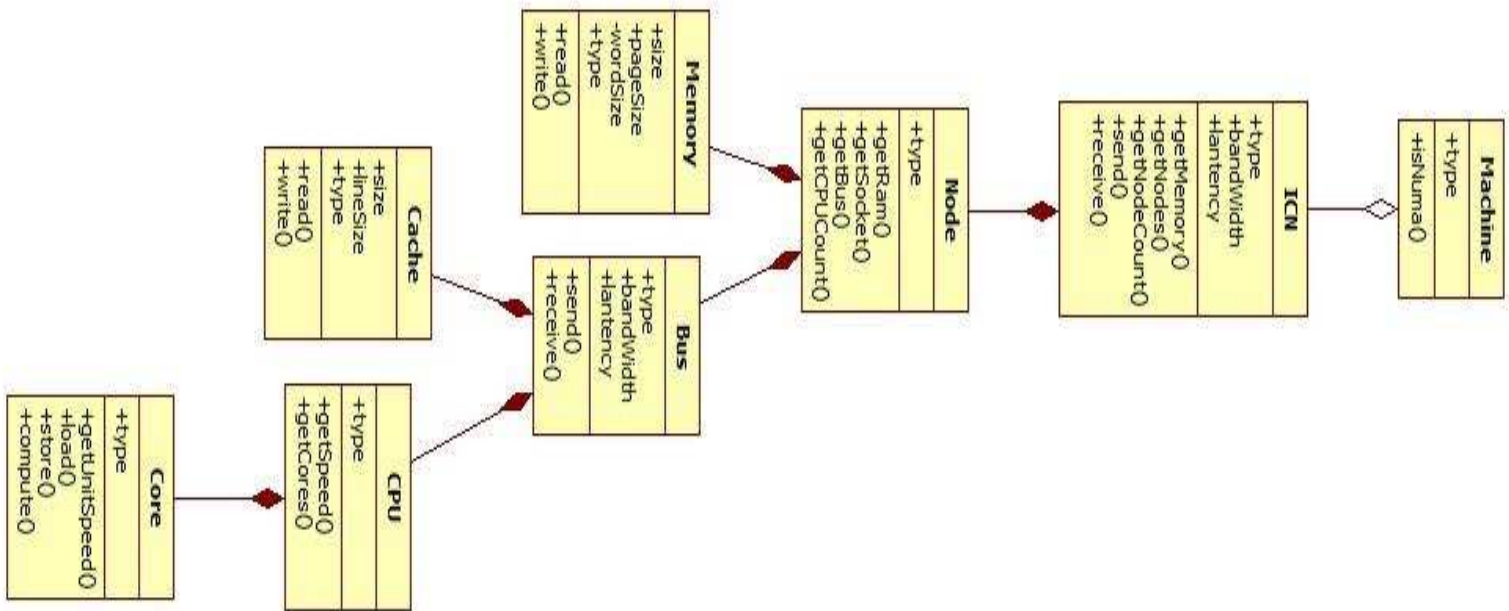


Figure 21.3: NUMA Simulator Class Diagram

Figure below is showing relationship between classes and interfaces, the structure of these relationships is so important, and it must be compatible with the topology of the addressed hardware. As we can see each class either implements the interface IHierarchical or extends another interface that implements it, so they inherit the same properties. This will help us identify parents and children in the hierarchy by calling getParent() and getChildren() and so we know they return IHierarchical objects in split of different types, for example, Cores are children of Caches in the hierarchy by calling isType().

Property of belonging is defined by the composition relations in most cases, we notice that IHierarchical interface has an extend relation with all other interfaces that will implement later by classes but each class has its own level as shown on Figure4.4. We did that on purpose to keep all resources collected and easy to reach.

We can see that Machine class is on top of all others classes and has aggregation relation with ICN because of existing of machines without ICN. There Force, doesn't mean that all other classes are the same. We can see down later on Figure4.7 that all other relations are composition because disappearance of one component affect others existing.

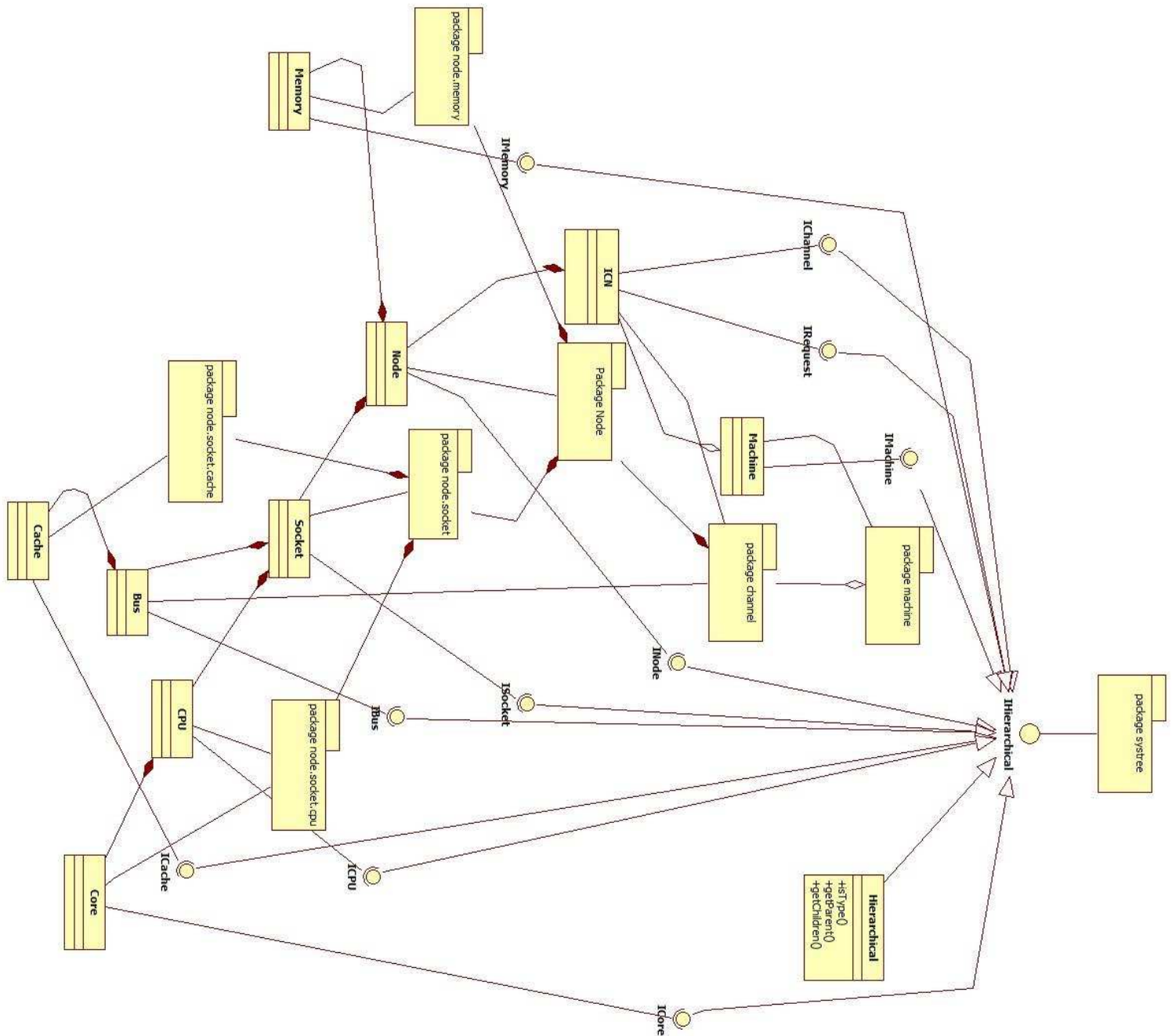


Figure 22.3: An overview of NUMA class diagram

Figure below is the resume of our modeling in one figure that shows the relationship between packages, classes and interfaces, with all the necessary information. The structure that we proposed here is so important to develop later on our simulator because it's compatible with the topology of the addressed hardware.

6) Sequence Diagram

The operators in sequence diagram are not necessary classes, it could be a module or even a

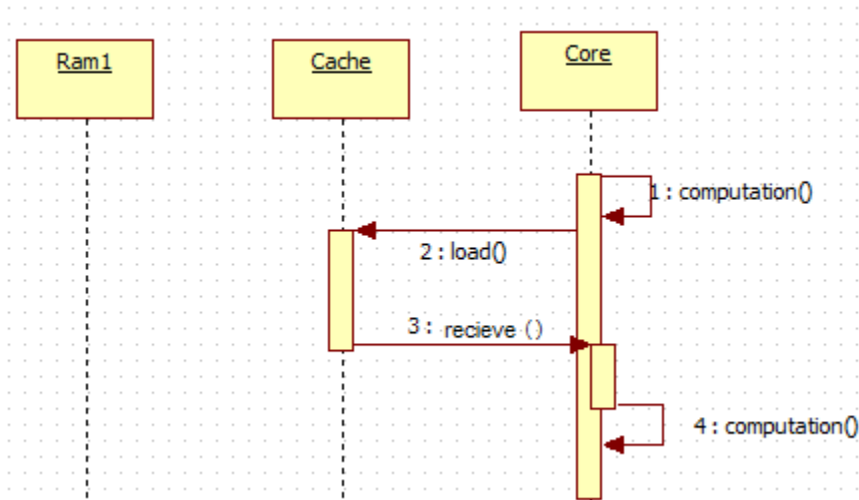


Figure 23.3: sequence diagram of local memory access to cache.

package. It will help us put more clarifications on the workflow.

It is possible to have communication between core and cache all the time and for so many reasons such as storing or loading small units of data.

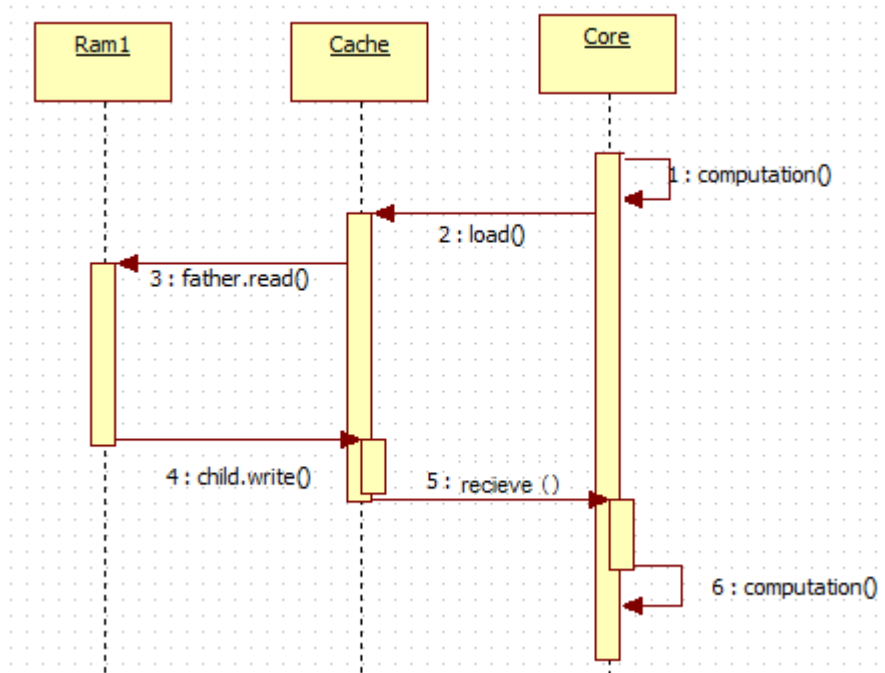


Figure 24.3: sequence diagram of local memory access to RAM.

As we can see on this two figures that core has different delay time accessing cache and RAM resources. Delay time here may not be considered or will be ignored when none local memory access arrived where time axes is so much bigger then what we see here.

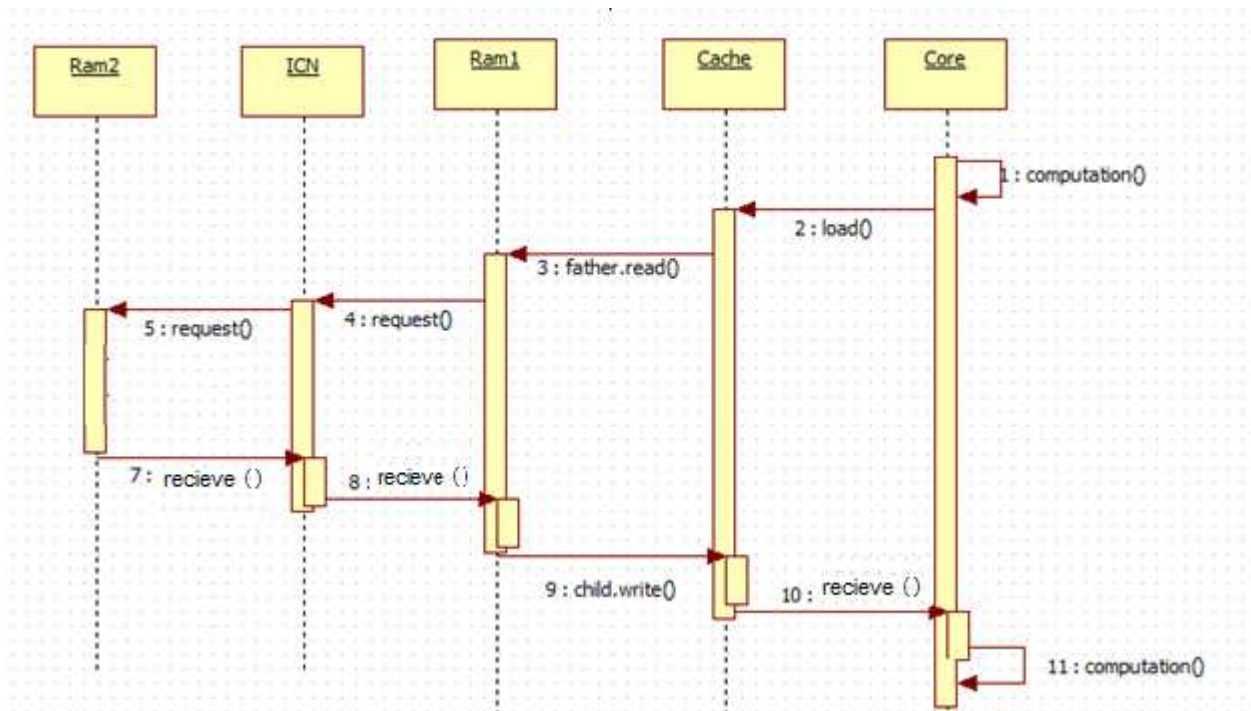


Figure 25.3: sequence diagram of remote memory access.

As we can see together, remote memory access has and always been the big loss of time on NUMA systems. RMA took a long time to return the data depending on which protocol the system is using to point or address the data on other nodes.

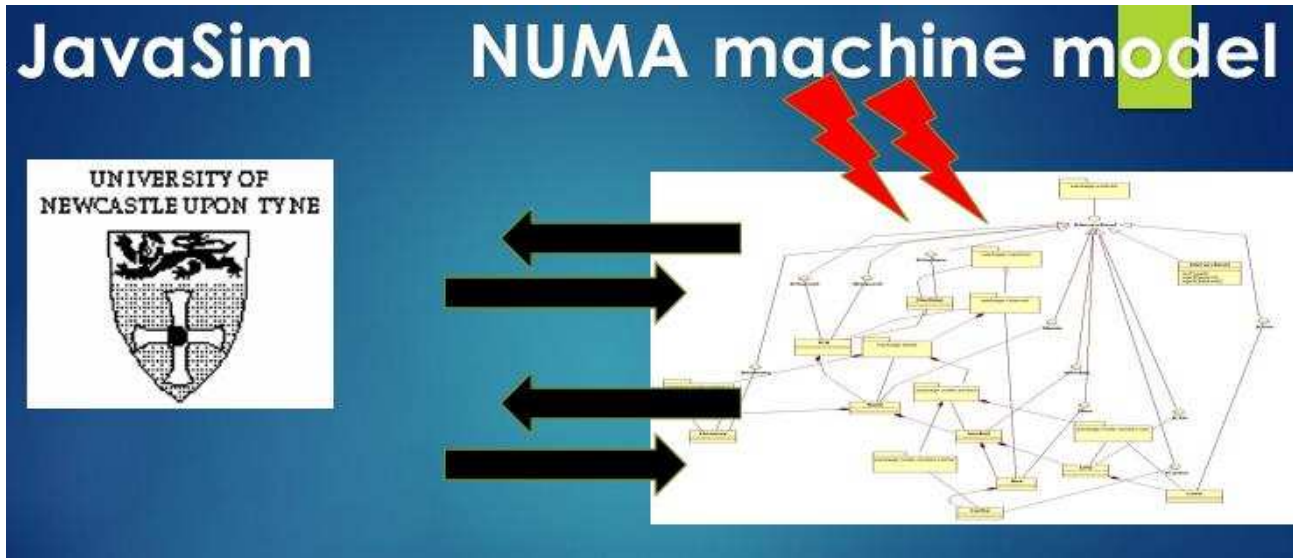


Figure 26.3: simulation process of executing a task on a single core

Figure shows the general process of communication and computation simulation of our model.

7) Simulating Approach:

7. 1) Introduction:

After we have seen together how we designed and modeled our NUMA machine. We will walk you through the simulation model, with very specifics and interesting title from existing simulators to the structure of simulator itself. Also, we will see what are the most important parts or sections in any simulator in general.

Simulation model will talk the NUMA model and the history of this talk are events of this simulation approach

7. 2) Existing simulator

An up to date listing of all the available simulation software. By listing the one that are based on Java in one form or another: CSIM, DESMO-J, DEUS, DSOL, JavaSim, JiST, JSL, SimJava, Simkit, and Tortuga.

For reasons of documentation and familiarity we have choose JavaSim to be our simulator and be our talk in this second section of the chapter. Where we will see more details about structure and implementation.

7. 3) Basic simulation object

7. 3. 1) Simulation object

The overall simulation model. Controls reading of input data, starting of the simulation run, termination of the simulation run, and printing of output reports.

7. 3. 2) Event manager object

Maintains simulated time and the list of future and conditional events to be executed.

7. 3. 3) Entity object

The basic object for the simulation. Can be permanent or temporary during the simulation run and can be either active or passive in the model logic.

7. 3. 4) Process

A sub-class of thread that allows an entity to execute a series of methods in simulated time while other entities execute their own series of methods.

Note that we have separated the two objects Entity and Process. When an Entity is playing an active role in the simulation, it will have one or more Processes underway. When it is playing an inactive role or is temporarily dormant, it will have no Process underway.

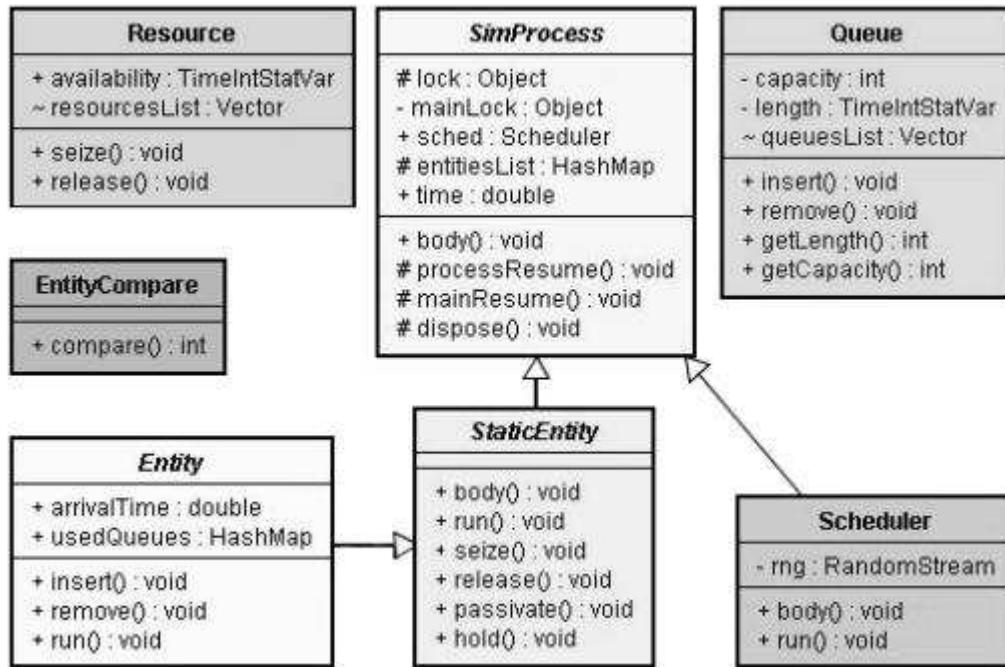


Figure 27.3:preview of Class diagram of basic simulator library of JAVASIM

As we can see, this example shows us that the package is made up of classes dealing with active entities, scheduler, and queues and resources. The coroutine like mechanism is implemented through **SimProcess**, **Scheduler**, **StaticEntity** and **Entity** classes.

7. 4) Javsim:

7. 4. 1) Why javasim Simulator!

javaSimulation is Java package for process-based discrete event simulation. The package is a Java implementation of the simulation facilities provided by SIMULA. In addition to the simulation facilities, the package also includes the facilities for list manipulation, random number drawing and coroutine sequencing as found in SIMULA

A simulation encompasses a set of interacting processes. A process is an object associated with a sequence of activities ordered logically in simulated time. Each process has its own life cycle and may undergo active and inactive phases during its lifetime. Processes represent the active entities in the real world, e.g., customers in a supermarket.

7. 4. 2) How javasim works:

In JavaSimulation the **processes** are described in one or more **subclasses** of the class **Process**. Their life cycles are described by overriding the abstract method actions.

```

class Request extends Process {
    public void actions() {
        // The life cycle of a customer
    }
}

```

An outline of class **Processes** shown below

```
public abstract class Process extends Link {
    protected abstract void actions();

    public static double time();
    public static void activate(Process p);
    public static void hold(double t);
    public static void passivate();
    public static void wait(Head q);
}
```

7. 4. 3) Entities basic methods

Entities can start processes and schedule activities using the basic methods such as:

scheduleWait(dur, pri):

Stops the execution of the current method for the given duration in simulated time. Can be placed anywhere within a method and can be used multiple times within a method.

startProcess(method, arg1, arg2, ...):

Calls the given method with the specified arguments, i.e. this.method(arg1, arg2, ...); However, it differs from a simple method call in that a new process is created, allowing the method to be executed in parallel to the original method, rather than in series.

scheduleProcess(dur, method, arg1, arg2, ...):

The same function as startProcess except that the given method is called after the specified delay. The new process is created at the end of the delay, minimizing the number of active processes.

while(condition) { waitUntil() ; } waitUntilEnded():

Code structure used to create conditional waits. Two new methods waitUntil() and waitUntilEnded() are used. Stops execution of the current method until the given condition is false.

scheduleLast():

Stops execution of the current method until all other events scheduled for the present simulated time have been executed.

getProcess():

Returns the active process executing the method. The methods interruptProcess and killProcess are the only ones that require a process to be identified.

InterruptProcess(processName):

Interrupts the given process and causes its next event to be executed immediately.

killProcess(processName):

Interrupts the given process and terminates it.

The simulation engine allows both the process- and event-orientation to be used freely in the

construction of a simulation model:

- The method `scheduleWait` is the key one for writing a process-orientated simulation model. Unlike traditional programming languages for writing process-oriented simulations, we have made the distinction between starting a new method, which is done in series with the original method, and starting a new process (using the method `startProcess`), which is done in parallel with the original method. Traditional simulation languages return control to the original method if the called method is halted by a wait, which is equivalent to starting a new process each time a method is called.

- The method `scheduleProcess` is the key one for writing an event-oriented simulation model. It is somewhat more efficient than using the methods `startProcess` and `scheduleWait` because it delays the creation of a new process until the end of the wait.

With a simulation engine of this type, discrete-event simulation becomes a simple extension of the Java programming language.

8) Simulator and model relationship

8. 1) Processors placement on scheduler queue

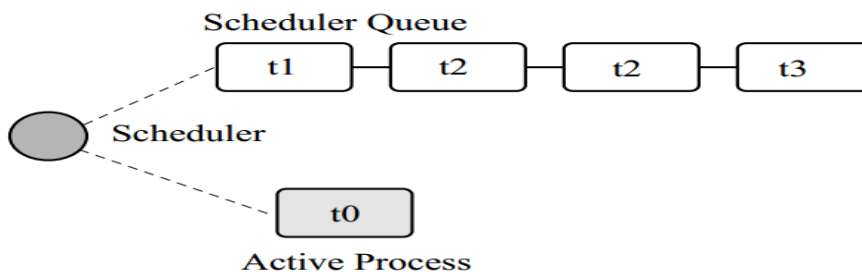


Figure 28.3: Scheduler process interaction

As Figure shows, the scheduler co-ordinates the entire simulation run, effectively monitoring the active and passive processes to enable it to determine when, and which, process to activate next. A simulation application cannot affect the scheduler directly, but can do so only indirectly through modifications of the scheduler queue.

8. 2) Scheduler class interface

The scheduler is an instance of the **Scheduler** class. It is the responsibility of the application programmer to ensure that only a single instance of this class is created. The scheduler maintains the simulation clock, and the current value of this clock is obtained by invoking the **CurrentTime()** method.

```
public class Scheduler
{
    public static double CurrentTime ();

    public static synchronized void reset ();
    public static synchronized boolean simulationReset ();

    public static synchronized void startSimulation ();
};
```

8. 3) Simulation processes

As was described in the previously, JavaSim supports the process-oriented approach to simulation, where each simulation entity can be considered a separate process. Therefore in JavaSim the entities within a simulation are represented by process objects. These are Java objects which have an independent thread of control associated with them at creation time, allowing them to convey the notion of activity necessary for participating in the simulation.

In keeping with the object-oriented paradigm, and to make development of process objects simpler, classes inherit the process functionality from the appropriate base class (**Process**). This class defines all of the necessary operations for the simulation system to control the simulation entities within it, and for them to interact with it and each other.

At any point in simulation time, a process can be in one (and only one) of the following states:

- **Active:** the process has been removed from the head of the scheduler queue and its actions are being executed.
- **Suspended:** the process is on the scheduler queue, scheduled to become active at a specified simulation time.
- **Passive:** the process is not on the scheduler queue. Unless another process brings it back on to the queue it will not execute any further actions.
- **Terminated:** the process is not on the scheduler queue and has no further actions to execute. Once a process has been terminated it cannot be made to execute further in the same simulation run.

A process which is either active or suspended is has to be scheduled to obtain the time of computation and communication of specific resources and furthermore to build the statistics in which we aimed.

8. 4) Illustration

To illustrate how a simulation process could be implemented from the **Process** class we shall consider the example of a queue of request arriving at ICN. For this example, this involves three classes:

- **ICN:** instances of this class represent the customers in the queue.
- **Queue:** the instance of this class (queue) is the queue into which request are places.
- **Arrivals:** this is the process which creates new ICN for insertion in queue.

The implementations of the **Request** and **Queue** classes are not important to this example. The implementation of the **ICN** class could be:

Class **ICN** extends **Process**

```
{  Public Request () {};  
  void Request ()  
  { for (;;)   
  {  Request r = new Request ();  
    queue.insert(c);  
    Hold(20.0);  
  } }
```

This class is just an example of how the relationship would be later on implementation chapter.

9) Summary:

This chapter can be described as an overview of the modeling and designing process of an existing or physic machine. We have given some hypotheses for technical aspects. Also, we have seen how to split a complex problems by setting up small pieces then collected on one big complex system.

A better choice of a data structured and good design can lead to a better simulator with an extendibility and ability to be easy to reuse. It is clear that too many options are available, and researches are on the go since NUMA systems were introduced either by updating the existing simulator on NUMA machine or by creating new ones to study new algorithms that are NUMA systems.

In our research project, we have study some well-known simulators to work with our modeling NUMA machine, not only that, we managed to combine multiple choices of data structured and creating a hierarchical system to control our machine. All this have been done to study the result of their execution. These approach will be described in next chapter.

1. Introduction:

What we are going to see in this chapter come as a result of all the researches and studies of what we have seen in the previous chapters.

Unfortunately, to face the lack a real NUMA machine to compare our results and to carry on with our research, we decided to use a simulator. However, the existing multicore machine simulators are mostly low level simulators and they are more suitable for studies such energy consumption or physical memory allocations and so on, but none of them can be good for our study, as NUMA machines are trending in multiprocessors world, and there is no such ideal machine simulator. Therefore, we had no choice but to design our own simulator built on our specification and needs to make it as appropriate ad possible for our study.

It clear now after all what we have discussed in previous chapters, about NUMA machine and architectures models. To conclude that we need hierarchical system to control our model, but it's not good to use simple data structure like List or ArrayList or Map. So, we build a hierarchical system based on interfaces. We didn't stop there only but we add more to class interfaces to control events and more in details in this chapter.

2. NUMA machine Implementation:

We all talk about the way that we do things in our everyday work, hobbies, and home life and recognize repeating patterns all the time.

We see the same thing in programming, when we tell a colleague how we accomplish a tricky bit of programming so that the colleague doesn't have to recreate it from scratch. We simply recognize effective ways for objects to communicate while maintaining their own separate existences.

We have three main parts to talk about in this section, we have face in our design process. Our aim is to create a library to be used later in computer science communities. So, we focus on design pattern programming in our project.

2.1. Interfaces:

Interfaces are core part of java programming language and used a lot not only in JDK but also java design patterns, most of the frameworks and tools. Interfaces provide a way to achieve abstraction in java and used to define the contract for the subclasses to implement.

After we learn the Important Points about Interfaces in Java. We find no excuse to not use it. It was such a really relief later on in programming process. One of the most important help provide by interfaces is organization. One we used interfaces it become less pain to check all methods in each class, we just create new methods whenever we need new one and method will be add to each class implement this class interface. We could implement as much as we need interface classes in opposite of extending classes.

```
14  L  */
15  public class Interconnection implements IInterConnection, IChannel {
16
17  L  /**
18  L  *
19  L  */
20  public static int level = 0;
21
22  L
```

Figure 29.4: example des Implements des interfaces.

Interconnection class implement two different class interface without any problems in opposite of extends which you have to extends only one class.

2.2. Extensibility on model using interfaces:

Using interfaces mean that we can extend any component class any time we want without changing related classes.

In our model we have supposed some hypnotizes because of manner of time we could not build it. So, what if someone else want to use our model but by changing the interconnection class.

```
15  public class Interconnection implements IInterConnection, IChannel {
16
17  L  public static int level = 0;
18
19  L  protected int bandWidth;
20
21  L  protected int lantency = 0;
22
23  L  private IHierarchical[] nodes;
24
25  L  private Request lRqt;
26  L  private Response lRsp;
27
28  L  public Interconnection(IHierarchical[] nodes, int lantency) {
29  L  L  this.lantency = lantency;
30  L  L  this.nodes = nodes;
31  L  }
32
33  L
```

Figure 30.4: First interconnection class

In order to use this technique, all what we need to do is to re-implement the IInterconnection class interface and re-use it without changing any other classes. In opposite of classical programming technique.

This technique go furthered to parameters of functions. Where we made it universal by giving each functions list of IHierarchical class interface. It allow any user to create any topology.

```

17 public class Interconnection2 implements IInterConnection, IChannel {
18     protected int bandWidth ;
19
20     protected int lantency = 90;
21
22     private IHierarchical[] nodes;
23
24     private Request lRqt;
25     private Response lRsp;
26
27     public Interconnection2 (IHierarchical[] nodes){
28         this.nodes = nodes;
29     }
30

```

Figure 31.4: Second interconnection class

2.3.Hierarchical:

After we work with interfaces in first place. It became easier to manipulate the entire project as one piece. Because our NUMA machine based on hierarchical concept, we find no other way but to use it. So we use it to build our hierarchical system by implementing IHierarchical class interface into all our classes and also IChannel class to help with the exchange of information between components of machine.

```

1 package systree;
2 import java.util.ArrayList;
3
4 /**
5  *
6  * @author zaya
7  */
8 public interface IHierarchical {
9     public final ArrayList<INumaEventListener> numaEventListners = new ArrayList<>();
10    public void addNumaEventListener(INumaEventListener numaEventListener);
11
12    public IHierarchical getParent();
13
14    public ArrayList<IHierarchical> getChildren();
15
16    /**
17     *
18     * @return
19     */
20    public int getIdentity();
21
22    /**
23     *
24     * @return
25     */
26    public String isType();
27
28    public void fireEvent(EventAdapter args);
29
30 }

```

Figure 32.4: IHierarchical class interface

So we used our interfaces again to implements more methods such as SEND() and RECEIVE() in each component, in order to achieve the connectivity between components and make our model more extensible. To exchange data (request and response) between machine resources.

3. Events based model:

Designing a model that is totally independent from simulator was such hard work but because we worked in first place with class interfaces. We had used an event fire mechanism in our model to be listened by the simulator.

The mechanism that we are using here, it called Observer / Observable design pattern technique, which in NUMA model Observable (events fire) is each component and observer is JAVASim, the listener of events.

This event is simple method in our IHierarchical interface class (extends by IMachine) that has as an argument of another class interface which is the event listener class. As shown in figure below each resources has multiple events in order to do the statics, this part is coming later on in next section.

```
20 public class Machine implements IMachine, IChannel {
21
22     public static IHierarchical root;
23
24     private Interconnection[] interConnection ;
25
26     // TODO event listener component
27     @Override
28     public void addNumaEventListener(INumaEventListener numaEventListener){
29         if(!this.numaEventListeners.contains(numaEventListener))
30             this.numaEventListeners.add(numaEventListener);
31     }
```

Figure 33.4: addNumaEventListener to fire events from component.

```
1
2 package systree;
3
4 /**
5  *
6  * @author zaya
7  */
8 public interface INumaEventListener {
9     public void numaEventHandler(EventAdapter args);
10 }
11
```

Figure 34.4: INumaEventListener to handle events and pass it to listener.

As we could see on this previous figures that machine class as an Override method addNumaEventListener which is implemented from IHierarchical class (extends from IMachine) to add numaEventListener as an argument of type INumaEventListener. With those simple steps we will be able to fire new events to INumaEventListener class, which we will see later how to capture it in simulator.

One of the main problem that faces as also, is how to catch the end of the event. We add at the end of this event another fire event to catch the end of each event.

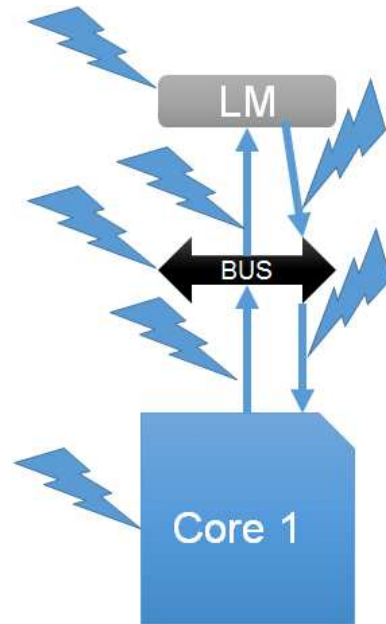


Figure 35.4: events fire in each resources

4. Simulator integration:

In JavaSim, simulation processes are managed by a scheduler and are placed on a scheduler queue (the event list). Processes are executed in pseudo-parallel, i.e., only one process executes at any instance of real time, but many processes may execute concurrently at any instance of simulation time. The simulation clock is only advanced when all processes have been executed for the current instance of simulation time.

Inactive processes are placed on to the scheduler queue, and when the current active process yields control to the scheduler (either because it has finished or been placed back onto the scheduler queue), the scheduler removes the process at the head of the queue and re-activates it. When the scheduler queue is empty, i.e., there are no further processes left to execute, the scheduler terminates the simulation.

The scheduler co-ordinates the entire simulation run, effectively monitoring the active and passive processes to enable it to determine when, and which, process to activate next. A simulation application cannot affect the scheduler directly, but can do so only indirectly through modifications of the scheduler queue.

In order to achieve the global goal to create “NUMA simulator”, JAVASIM become the driver for this NUMA model. We have create communication mechanism between them to facilitate the driven and to exploit the statistics later on.

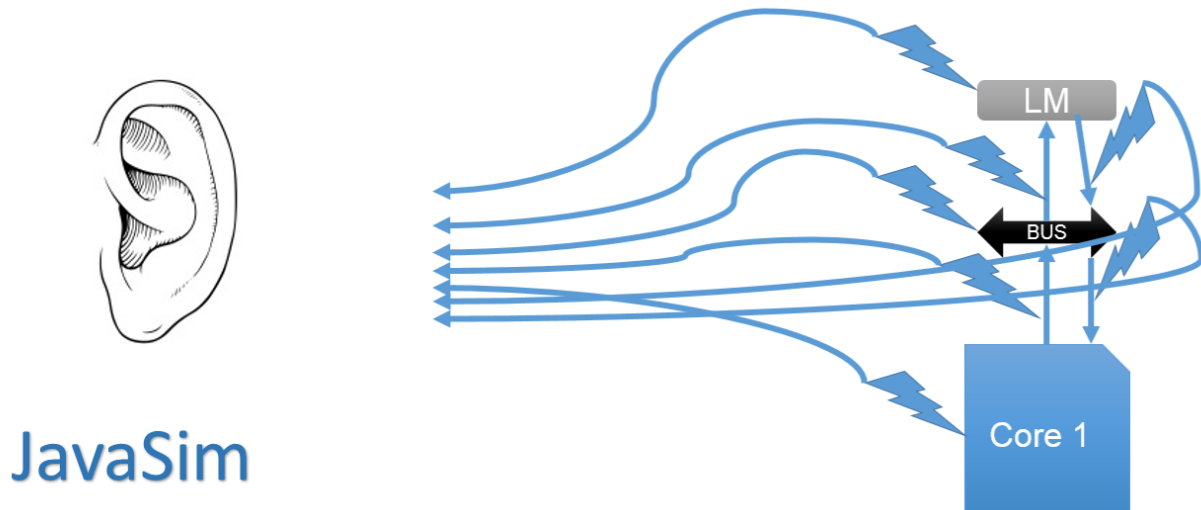


Figure 36.4: JAVASim listening to NUMA machine event fire

```

32 public class ClassTest implements INumaEventListener{
33     public void addEventListenerToComponent(numa.sim.ClassTest test){
34         RelationClass rC = new RelationClass();
35         ArrayList<IHierarchical> all = rC.getAll(test.mach);
36         for(IHierarchical one : all){
37             one.addNumaEventListener(test);
38         }
39     }
40     @Override
41     public void numaEventHandler(EventAdapter args) {
42         Logger.getLogger(ClassTest.class.getName()).log(Level.INFO, "Method : "+ args.eventName + " Message: "+ args.message+" enti
43         evtntlist.add(args);
44         if(!args.message.contains("done")){
45             evtntarray.add(new EventStartEndAdapter(args));
46         }else{
47             Collections.reverse(evtntarray);
48             for(EventStartEndAdapter ea : evtntarray){
49                 if(ea.start.component.getIdentity() == args.component.getIdentity()
50                    && ea.end == null && args.message.contains( ea.start.message)){
51                     ea.end = args;
52                 }
53             }
54             Collections.reverse(evtntarray);
55         }
56     }
57     public ArrayList<EventAdapter> evtntlist = new ArrayList<>();
58
59

```

Figure 37.4: example of Class is listening to events

5. Topology Loader

It is responsible for creating the object model for the topology to run the simulation on and showing it in the topology viewer. After the user structured the topology, an instance of Machine will be created. The user has to generate tasks and map data then called run() method in Machine class to start processing the tasks. This is the class responsible for building the object model and simulating the topology.

```

88     Node[] nodes = new Node[2];
89     children = new ArrayList<>();
90     children.add(new Socket(bus));
91     children.add(new Socket(bus1));
92     children.add(ram);
93     nodes[0] = new Node(new Bus(children, 1000,26));
94
95     //-----
96     ArrayList<task.ITask> task1 = new ArrayList<>();
97     task1.add(new Communication(0,ram1));
98     task1.add(new Computation(20));
99     task1.add(new Computation(20));
100    ArrayList<ArrayList<task.ITask>> tasks1 = new ArrayList<>();
101    tasks1.add(task1);
102    Core[] cores2 = new Core[1];
103    cores1[0] = new Core(tasks1);
104    Cache cache2 = new Cache();
105    children = new ArrayList<>();
106    children.add(new CPU(cores2));
107    children.add(cache2);
108    Bus bus2 = new Bus(children, 1000, 14);
109    children = new ArrayList<>();
110    children.add(new Socket(bus2));
111    children.add(ram1);
112    nodes[1] = new Node(new Bus(children, 1000,26));
113    //-----
114    Interconnection[] icn = new Interconnection[1];
115    icn[0] = new Interconnection(nodes, 25);
116
117    mach = new machineNuma.Machine(icn);
118    mach.Run();

```

Figure 38.4: Machine loader

Each component of the topology has its own representing class that is implementing IHierarchical class which is responsible on hierarchical of machine. Also, each component has its own interface class that implement different methods depends on component type.

6. Task & Data

Threads are represent as tasks in this model. Each task has multiple subtasks which can be type of communication or computation. Each task requires execution time, parameter and data location with address to be scheduled manually. Then, the core cast each subtask to its original type. Which we have two types (Computing, Communication).

Each memory bank has bunch of data stored already in it when the machine start running. With is mapped by user in our case using RamAdapter class.

```
ArrayList<RamArchitecture> dataRam = new ArrayList<>();
dataRam.add(new RamArchitecture(0, "data from Memory"));
dataRam.add(new RamArchitecture(1, "mjqsdklhfjqmskdjfkmqj"));

Memory ram = new Memory(dataRam, 100);

ArrayList<RamArchitecture> dataRam1 = new ArrayList<>();
dataRam1.add(new RamArchitecture(0, "datat"));
dataRam1.add(new RamArchitecture(1, "mjqsdklhfjqmskdjfkmqj"));
dataRam1.add(new RamArchitecture(3, "data from Memory1"));
dataRam1.add(new RamArchitecture(4, "data 4 from memory 1"));

Memory ram1 = new Memory(dataRam1, 100);

ArrayList<task.ITask> task = new ArrayList<>();
task.add(new Computation(3));
//task.add(new Communication(0, ram));
task.add(new Computation(20));
task.add(new Computation(20));
task.add(new Communication(3, ram1));
```

Figure 39.4: data mapping & tasks scheduling

7. Simulation Scenario

Let's take a look to the progress of the simulation and understand how the previous components work together along with other components to do the emulation, two important components are cores and memories. Figure below shows a general overview about the workflow of the simulator, this will also help us evaluate the results after execution, furthermore, doing more updates by adding more events to collect more information.

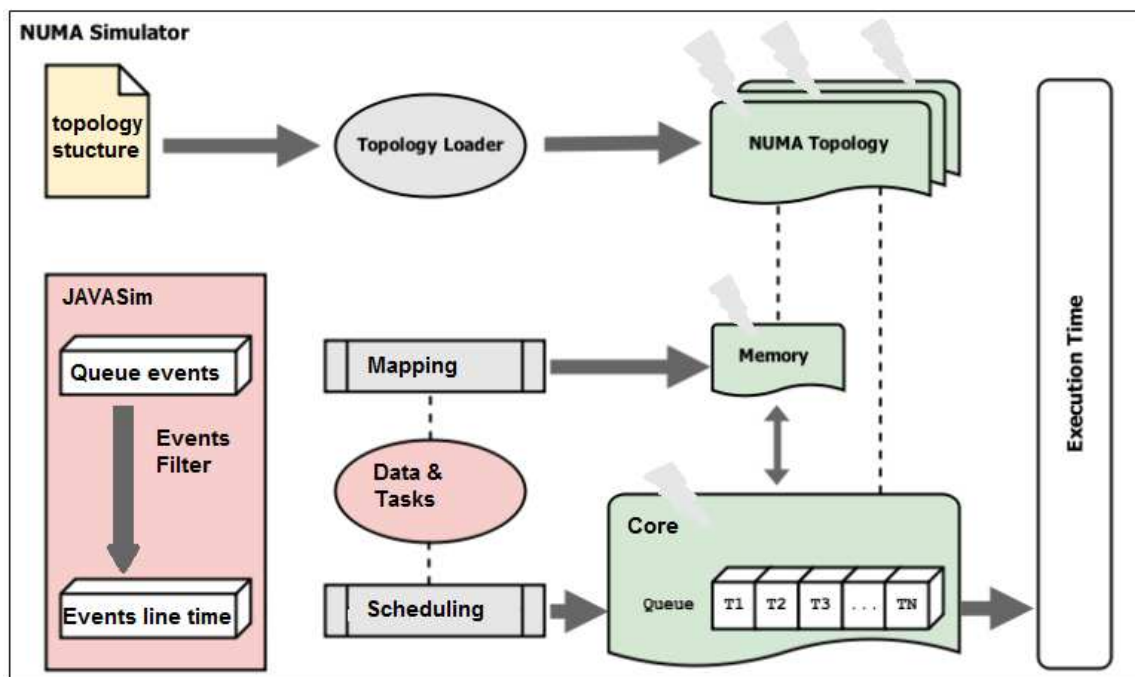


Figure 40.4: NUMA simulator workflow

Loading the topology and generating are two distinct operations, they only work together at the phase of scheduling, to do that, the simulator considers two objects from NUMA topology object model, Memory and core objects.

There is not much to be said about memories, they belong to a NUMA node and waiting for data to be mapped. As for process units, they are where the execution begins and ends.

8. Experiments:

Our experiments will give us a lot of information about the machine. But because time is so small, we expand the time from milliseconds to seconds just to show the statistics on chart and diagrams.

8.1.Scenario 01:

Figure in right side will be our topology for this experiment, which have two nodes with one socket in each, and two cores in each socket.

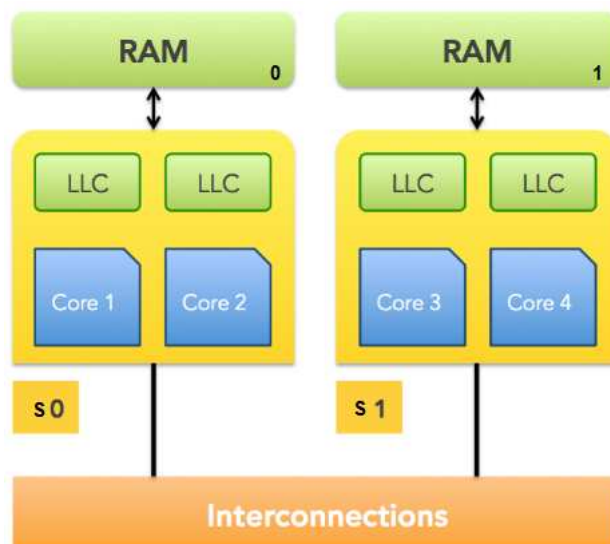


Figure 41.4: topology of two nodes and two cores

Mapping and scheduling:

Data: Ram should be mapped manually as show below.

```
ArrayList<RamArchitecture> dataRam = new ArrayList<>();
dataRam.add(new RamArchitecture(0, "data from Memory 0"));
dataRam.add(new RamArchitecture(1, "data1 from Memory 0"));

Memory ram = new Memory(dataRam, 100);

ArrayList<RamArchitecture> dataRam1 = new ArrayList<>();
dataRam1.add(new RamArchitecture(0, "data 0 from Memory 1"));
dataRam1.add(new RamArchitecture(1, "data 1 from Memory 1"));
dataRam1.add(new RamArchitecture(2, "data 2 from Memory 1"));
dataRam1.add(new RamArchitecture(3, "data 3 from Memory1"));
dataRam1.add(new RamArchitecture(4, "data 4 from memory 1"));

Memory ram1 = new Memory(dataRam1, 100);
```

Figure 42.4: mapping data

Tasks: Cores should be mapped manually as show below.

We schedule the same task in each of the two cores to simplify the code.

```
ArrayList<task.ITask> task = new ArrayList<>();
task.add(new Communication(0, ram));
task.add(new Computation(3));
task.add(new Computation(20));
task.add(new Computation(20));
task.add(new Communication(3, ram1));

ArrayList<ArrayList<task.ITask>> tasks = new ArrayList<>();
tasks.add(task);
tasks.add(task);
IHierarchical[] cores = new Core[1];
cores[0] = new Core(tasks);

IHierarchical[] cores1 = new Core[1];
cores[0] = new Core(tasks);
```

Figure 43.4: scheduling tasks

After we set up the topology, map the data on memories and schedule the tasks on cores. And with Observer / Observable technique we have all what the simulator need to extract the statistics into graph using open source library JFreeChart[16].

Chapter VI: Implementation and test

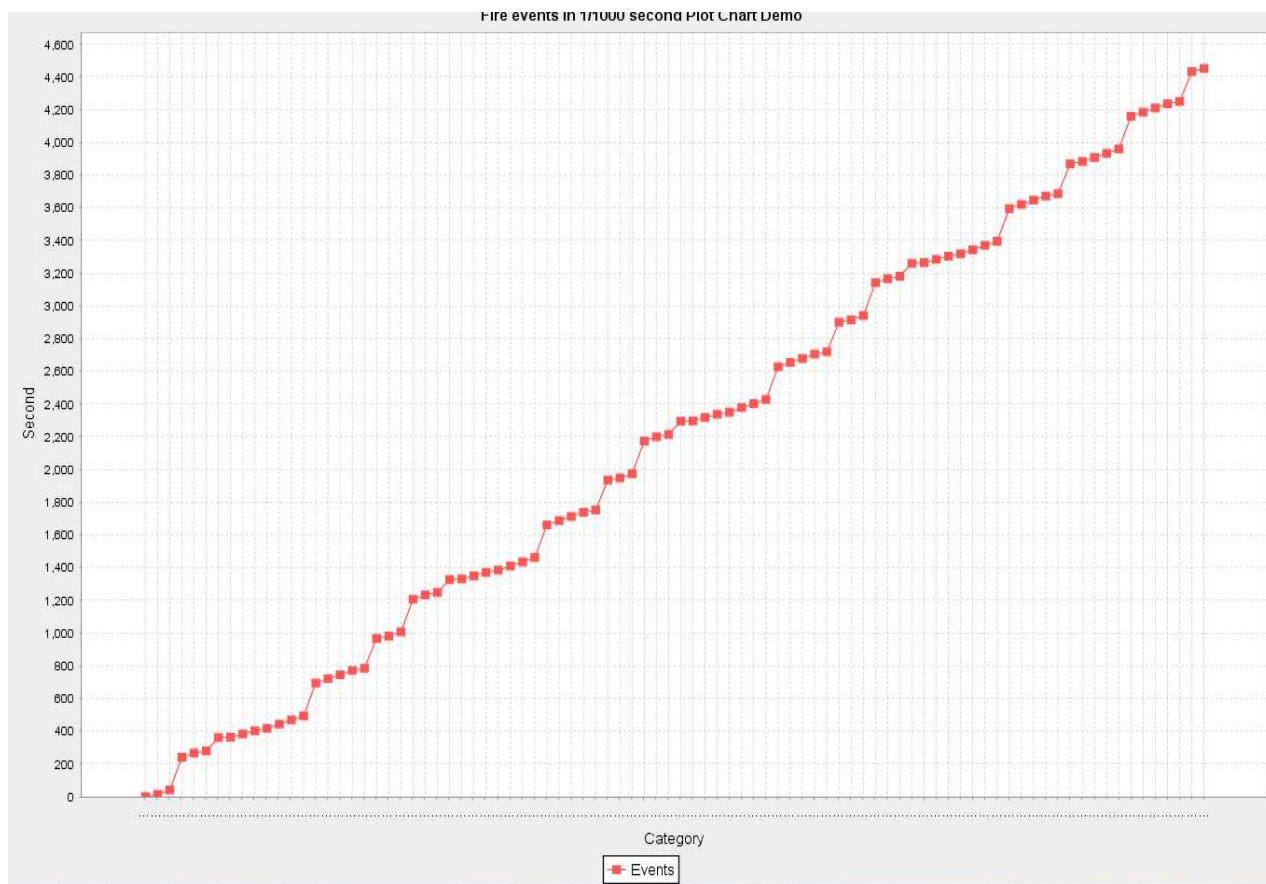


Figure 44.4: Event fire record in time as plot chart

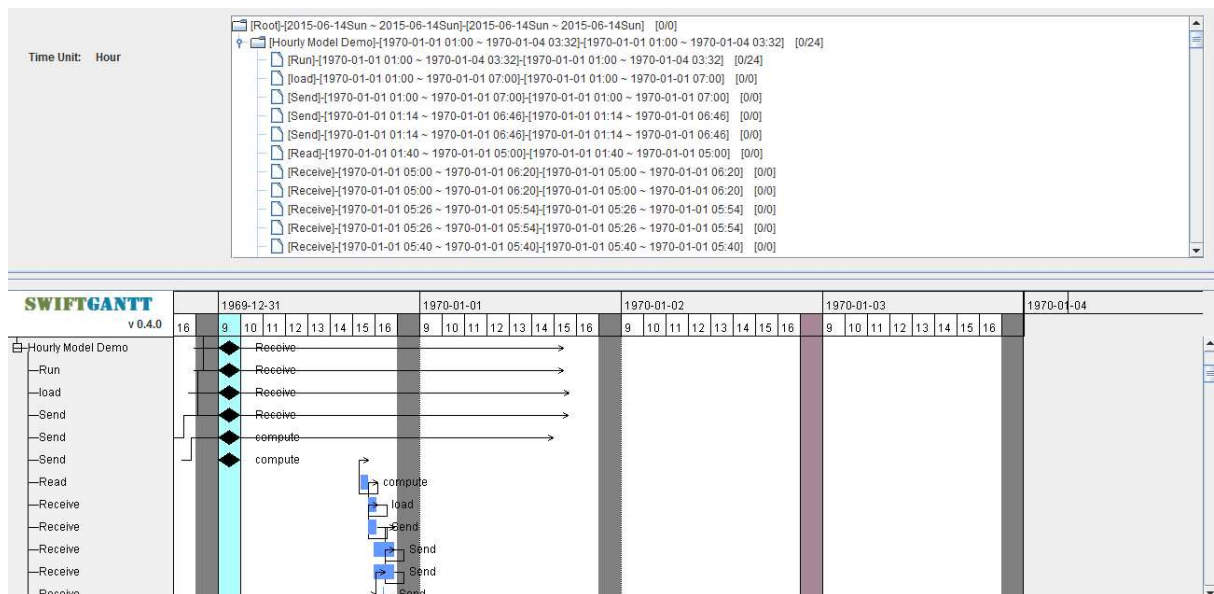


Figure 45.4: Gantt diagram[17] showing the schedule of events

8.2.Scenario 02:

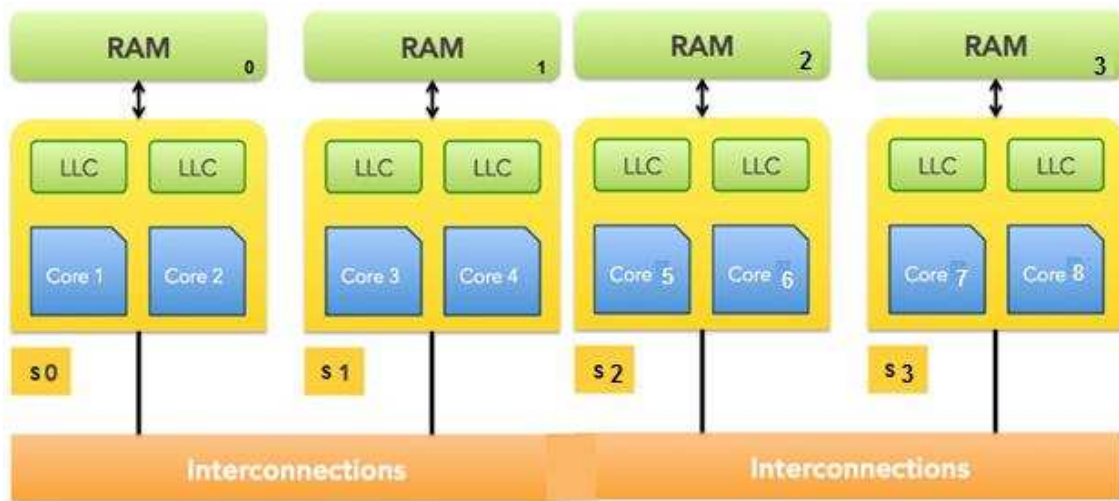


Figure 46.4: topology of four nodes and two cores

Figure above will be our topology for this experiment, which have four nodes with one socket in each, and two cores in each socket.

Mapping and scheduling:

Data: Ram should be mapped manually as show below.

```

143 public void do_scn002(){
144
145     ArrayList<RamArchitecture> dataRam2 = new ArrayList<>();
146     dataRam2.add(new RamArchitecture(0, "data from Memory 2"));
147     dataRam2.add(new RamArchitecture(1, "data1 from Memory 2"));
148
149     Memory ram2 = new Memory(dataRam2, 100);
150
151     ArrayList<RamArchitecture> dataRam3 = new ArrayList<>();
152     dataRam3.add(new RamArchitecture(0, "data 0 from Memory 3"));
153     dataRam3.add(new RamArchitecture(1, "data 1 from Memory 3"));
154     dataRam3.add(new RamArchitecture(2, "data 2 from Memory 3"));
155     dataRam3.add(new RamArchitecture(3, "data 3 from Memory 3"));
156     dataRam3.add(new RamArchitecture(4, "data 4 from memory 3"));
157
158     Memory ram3 = new Memory(dataRam3, 100);
159
160     ArrayList<RamArchitecture> dataRam = new ArrayList<>();
161     dataRam.add(new RamArchitecture(0, "data from Memory 0"));
162     dataRam.add(new RamArchitecture(1, "data1 from Memory 0"));
163
164     Memory ram = new Memory(dataRam, 100);
165
166     ArrayList<RamArchitecture> dataRam1 = new ArrayList<>();
167     dataRam1.add(new RamArchitecture(0, "data 0 from Memory 1"));
168     dataRam1.add(new RamArchitecture(1, "data 1 from Memory 1"));
169     dataRam1.add(new RamArchitecture(2, "data 2 from Memory 1"));
170     dataRam1.add(new RamArchitecture(3, "data 3 from Memory1"));
171     dataRam1.add(new RamArchitecture(4, "data 4 from memory 1"));
172
173     Memory ram1 = new Memory(dataRam1, 100);

```

Figure 47.4: Mapping data

Tasks: Cores should be mapped manually as show below.

We schedule the same task in each of the two cores to simplify the code.

```
175
176     ArrayList<task.ITask> task3 = new ArrayList<>();
177     task3.add(new Communication(0, ram));
178     task3.add(new Computation(3));
179     task3.add(new Computation(20));
180     task3.add(new Computation(20));
181     task3.add(new Communication(3, ram1));
182
183     ArrayList<ArrayList<task.ITask>> tasks3 = new ArrayList<>();
184     tasks3.add(task3);
185     tasks3.add(task3);
186
187     IHierarchical[] cores3 = new Core[2];
188     cores3[0] = new Core(tasks3);
189     cores3[1] = new Core(tasks3);
190
191
192     Cache cache3 = new Cache();
193
194     ArrayList<IHierarchical> children3 = new ArrayList<>();
195     children3.add(new CPU(cores3));
196     children3.add(cache3);
```

Figure 48.4: scheduling tasks

After we set up the topology, map the data on memories and schedule the tasks on cores. And with Observer / Observable technique we have all what the simulator need to extract the statistics into graph using open source library JFreeChart[16].

Chapter VI: Implementation and test

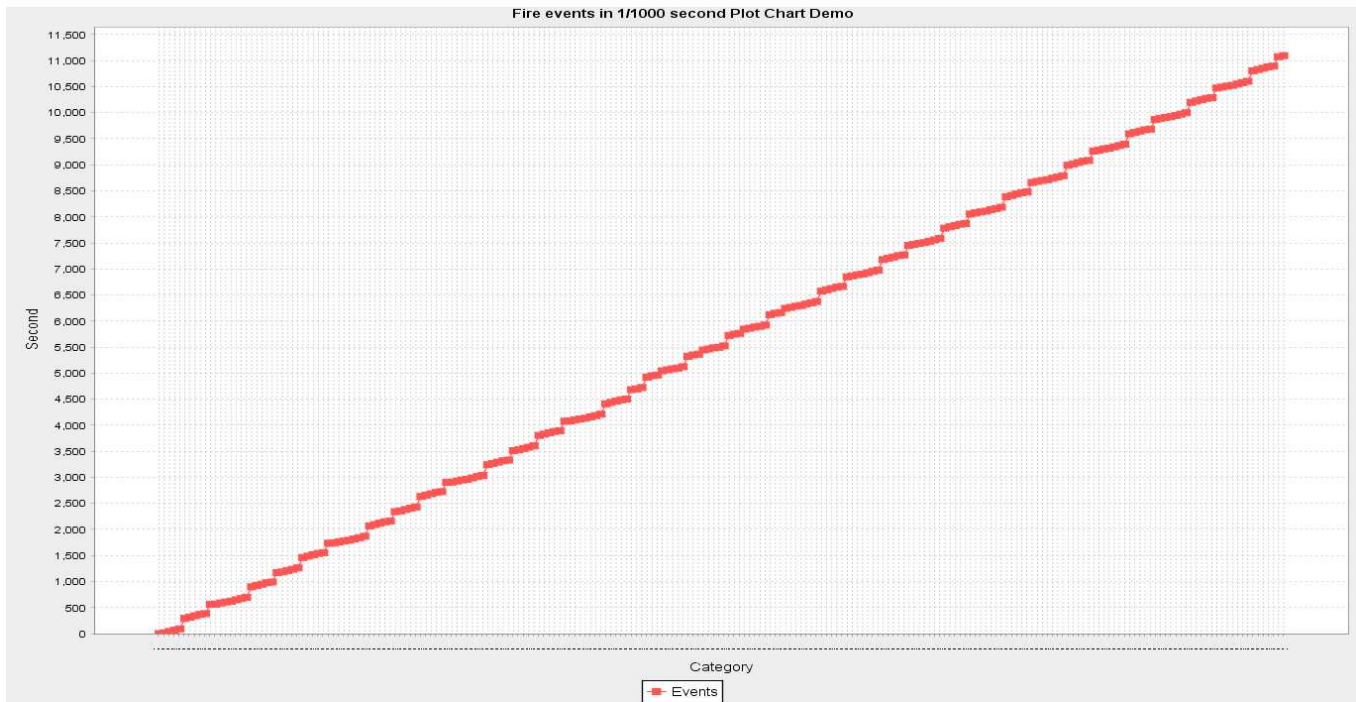


Figure 49.4: Event fire record in time as plot chart

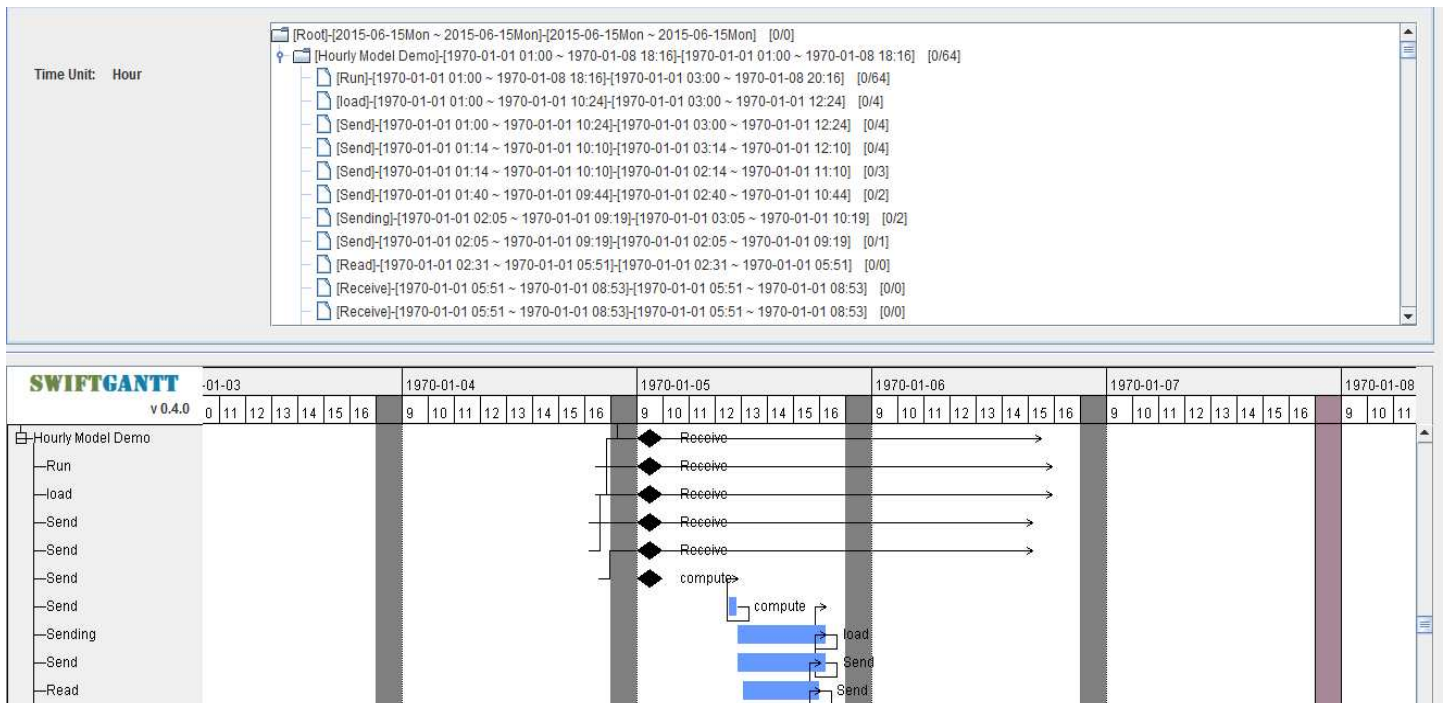


Figure 50.4: Gantt diagram[17] showing the schedule of events

9. Summary:

This chapter comes to endorse what we have seen in the previous chapters by resulting a software that simulates NUMA machines and configuration. To build this software, we have adopted HwLoc that provided us with real machines NUMA topologies and then constructing the object model of these topologies to run the simulation on.

After the conception of the topology and its implementation comes the generation of data and tasks, besides the topology, the simulator needs to know this as another information about the environment that the execution will be running on. At the execution phase, we have tested all those possible cases on different numbers of data sets and tasks, different topologies, and by evaluating the results we can say we have a better understand of NUMA environment.

Conclusion:

As NUMA machines are becoming widely spread many operating system schedulers are considered to work with NUMA platforms and take advantage of it. In this work we have focused on the fundamental aspects that could help the research in this field to update or build a perfect NUMA machine simulator.

As for the simulator, we can say that it's barely on its beta version, it has been built on our specifications and requirements to face the lack of real NUMA machines, and later on it is being adapted for general use. It simulates the real existing NUMA machines and their properties, referred to as topologies. The modeling took much of our time and focus, as it is indispensable for our experimentation and test phase.

Future Work

NUMA systems are a trend, they have been recently introduced and during our research we have deduced that such architecture is promising and researches are already on the verge of taking full advantage of NUMA. To do so, a dynamic scheduler can significantly optimize the execution time by finding a suitable combination of mapping and scheduling that fits the current configuration of the system with an intelligent manner. This doesn't preclude doing researches to design schedulers that are purely built for NUMA systems.

The analyses of techniques that utilize application characteristics are of little benefit if they cannot be applied in practical situations. To this end, more updates are to be added to the simulator to be as close as possible to a real state with **proofs**, one of these major updates is a real implementation of interconnections and its description that could affect the performance while performing remote access, we take for instance the latency and the bandwidth as important parameters to be considered to calculate the exact communication time.